Development of a Smart Autonomous Guided Vehicle Based Warehouse System with Integrated Robotic Arm for Material Picking and Transfer

A Project Report Submitted in Partial Fulfillment of the Requirements
for the Award of the Degree of

Bachelor of Technology

in

Automobile Engineering (Electric & hybrid vehicles)

Submitted by:

SHEIKH NOORUL A. USMANI, 22AEB180
ARBAZ RASHID, 22AEB485

Under the Guidance of:

Prof. MOHAMMAD MUZAMMIL
Principal, Zakir Hussain College of Engineering & Technology
Aligarh Muslim University

2025-26

Declaration

I hereby declare that this project report entitled *"Development of a Smart Autonomous Guided Vehicle Based Warehouse System with Integrated Robotic Arm for Material Picking and Transfer"* is the result of my own work and investigation, carried out under the supervision of Prof. Mohammad Muzammil, Department of Mechanical Engineering, Zakir Hussain College of Engineering & Technology, Aligarh Muslim Univeristy. This report has not been submitted elsewhere for the award of any other degree or diploma. All sources of information and literature used have been duly acknowledged.

Student Name: _____

Faculty No.: _____

Signature: _____

Date: _____

Certificate

This is to certify that the project report entitled *"Development of a Smart Autonomous Guided Vehicle Based Warehouse System with Integrated Robotic Arm for Material Picking and Transfer"* submitted by _____ (Faculty No. _____) is a bonafide record of work carried out under my direct supervision and guidance in partial fulfillment of the requirements for the degree of Bachelor of Technology in Mechanical Engineering during the academic year 2025–2026.

Project Guide:

_____

Designation: _____

Date: _____

Head of Department:

_____

Designation: _____

Date: _____

External Examiner: _____

ACKNOWLEDGEMENT

In the name of Allah, the Most Gracious, the Most Merciful.

First and foremost, all praise is due to **Allah (SWT)**, the Most Merciful and the Most Compassionate, for His endless mercy, guidance, and countless blessings in every aspect of life. By His will and grace, the author was granted the strength, patience, and ability to overcome challenges throughout this journey and successfully complete this project. Every step taken and every accomplishment achieved is only possible through His help and blessings.

The author extends his heartfelt appreciation to his parents, **Mr. Rashid Akber** and **Mrs. Kaniz Sakeena Rashid** for their unwavering love, sacrifices, prayers, encouragement, and constant support. Their guidance, patience, and belief have been a source of inspiration and strength throughout the completion of this work.

Special thanks are extended to **Ms. Rida Fatima** for her constant encouragement, patience, understanding, and unwavering support throughout this journey. Her presence, positivity, and words of encouragement provided strength and motivation during challenging times, and her continuous support played a meaningful role in maintaining confidence and determination throughout the completion of this project.

The author is deeply grateful to **Prof. Mohammad Muzammil** for his invaluable guidance, insightful suggestions, constructive feedback, and continuous encouragement. His mentorship and expertise played a crucial role in shaping and successfully completing this project.

Sincere appreciation is also extended to **Mr. Sheikh Noorul**, project partner, for his cooperation, dedication, and collaborative efforts throughout the development of this project. His support, teamwork, and valuable contributions greatly facilitated the successful accomplishment of the project objectives.

The author would also like to thank all faculty members of the Department of Mechanical Engineering for their guidance, encouragement, and support throughout this academic endeavor.

Finally, heartfelt thanks are extended to family members, friends, and well-wishers for their encouragement, understanding, and support, which provided constant motivation throughout this journey.

Abstract

This project presents the design, implementation, and analysis of a smart Autonomous Guided Vehicle (AGV) based warehouse automation system integrated with a Universal Robots UR3 robotic arm for autonomous material picking and transfer. The system combines a differential-drive mobile platform (ATLAS AGV) with a 6-DOF collaborative manipulator to form a complete end-to-end warehouse automation solution capable of navigating to shelf locations, identifying targets via RFID, executing pick-and-place operations, and returning to a home docking station.

The ATLAS AGV employs an 8-channel infrared reflectance sensor array for line-following navigation at 0.4 m/s, achieving sub-5 mm tracking accuracy under a Proportional-Derivative (PD) control law. Navigation decision-making is governed by a 12-state Finite State Machine (FSM) with RFID-based shelf confirmation at 21 floor-embedded tag locations. The UR3 arm operates on a five-layer ROS 2 architecture spanning robot description, Gazebo Harmonic physics simulation, ros2_control real-time control, MoveIt 2 motion planning (OMPL and PILZ planners), and the MoveIt Task Constructor (MTC) compositional task framework with a Robotiq 2F-85 parallel gripper.

The integrated system is implemented on ROS 2 comprising 11 packages and a PyQt5-based industrial control centre. Key results demonstrate 100% mission completion rate, $\pm 3^{\circ}$ turn precision, sub-5 mm line tracking accuracy, and complete autonomous operation without human intervention. A simulation-to-physical conversion pathway is documented, confirming that 70% of the control codebase transfers directly to physical hardware. The combined minimum viable build is estimated at approximately $800 with a production-quality build at approximately $1,250.

Keywords: Autonomous Guided Vehicle, ROS 2, Differential Drive, Line Following, PID Control, RFID, Pick-and-Place, MoveIt 2, Warehouse Automation, Industry 4.0.

Notations, Symbols, and Abbreviations

Symbols

Symbol	Description	Unit
v	Linear velocity of robot centre	m/s
ω	Angular velocity of robot	rad/s
v_L	Left wheel linear velocity	m/s
v_R	Right wheel linear velocity	m/s
ω_L	Left wheel angular velocity	rad/s
ω_R	Right wheel angular velocity	rad/s
r	Wheel radius	m
L	Wheel track width (separation)	m
R	Instantaneous turning radius	m
θ	Robot heading angle	rad
K_p	Proportional controller gain	—
K_i	Integral controller gain	—
K_d	Derivative controller gain	—
$e(t)$	Control error signal	—
$u(t)$	Controller output	rad/s
τ	Motor torque	N·m
I_{xx}, I_{yy}, I_{zz}	Principal moments of inertia	kg·m^2
m	Robot mass	kg
g	Gravitational acceleration	m/s^2
μ	Coefficient of friction	—
F_N	Normal force	N
α	Angular acceleration	rad/s^2
ζ	Damping ratio	—
ω_n	Natural frequency	rad/s
σ	Standard deviation	varies
Δs	Incremental distance	m

$\Delta\theta$	Incremental angle	rad
w_i	Sensor position weight	—
s_i	Binary sensor output	$\{0, 1\}$
$a, d, \alpha_{DH}, \theta$	Denavit-Hartenberg parameters	m, m, rad, rad

Abbreviations

Acronym	**Full Form**
AGV	Automated Guided Vehicle
AMR	Autonomous Mobile Robot
CPR	Counts Per Revolution
DDS	Data Distribution Service
DH	Denavit-Hartenberg
DoF	Degrees of Freedom
EKF	Extended Kalman Filter
FSM	Finite State Machine
GPIO	General Purpose Input/Output
GUI	Graphical User Interface
HMI	Human-Machine Interface
I2C	Inter-Integrated Circuit
ICR	Instantaneous Centre of Rotation
IMU	Inertial Measurement Unit
IR	Infrared
LiDAR	Light Detection and Ranging
LLM	Large Language Model
MTC	MoveIt Task Constructor
OMPL	Open Motion Planning Library
PD	Proportional-Derivative
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation
QoS	Quality of Service
RFID	Radio Frequency Identification
ROS2	Robot Operating System 2
RPM	Revolutions Per Minute
SAC	Soft Actor-Critic
SLAM	Simultaneous Localization and Mapping
SPI	Serial Peripheral Interface

TCP	Tool Centre Point
TF	Transform Frame
UART	Universal Asynchronous Receiver-Transmitter
URDF	Unified Robot Description Format
WMS	Warehouse Management System
YOLO	You Only Look Once

Contents

List of Tables

List of Figures

Introduction and Literature Review

Project Overview

Modern warehouses demand high-throughput, error-free material handling with minimal human intervention. Traditional warehouse operations, where workers spend 60–70% of their time on item retrieval walks, create significant operational inefficiencies [1]. The physical toll of repetitive walking, bending, and lifting in conventional order-picking environments contributes to worker fatigue, musculoskeletal disorders, and elevated error rates that compound during extended shifts. According to the Warehousing Education and Research Council, labour costs constitute approximately 65% of total warehouse operating expenditure, with order picking alone accounting for 55% of that labour allocation. These statistics underscore the pressing economic motivation for automated material handling systems.

Autonomous Guided Vehicles (AGVs) address these inefficiencies through continuous, precise material transport, while integrated robotic arms extend AGV capabilities to the physical manipulation of inventory items. The combination of guided mobile navigation with dexterous arm manipulation represents a complete material-handling automation solution aligned with the goals of Industry 4.0. Unlike standalone AGVs that merely transport entire shelving units (as in the Amazon Kiva paradigm) or stationary robotic arms that can only reach items within a fixed workspace, an integrated mobile manipulator combines the reach advantages of mobility with the dexterity of a multi-degree-of-freedom arm. This synergy enables individual item-level picking from arbitrary warehouse locations—the so-called "last metre" problem that remains unsolved by conventional AGV-only or arm-only solutions.

This project presents the development of the ATLAS Smart Warehouse AGV, a ROS 2-based autonomous mobile robot integrated with a Universal Robots UR3 6-DOF collaborative arm and a Robotiq 2F-85 parallel gripper. The system is capable of: receiving pick missions via a control GUI; navigating autonomously to target shelf locations using line-following and RFID identification; executing structured pick-and-place operations using motion-planned arm trajec-

tories; and returning to a home docking station. The entire system operates within a simulated Gazebo warehouse environment with a clearly documented pathway for physical hardware deployment.

The engineering philosophy underlying this project is simulation-first development with hardware-ready architecture. Rather than developing directly on physical hardware—where debugging is slow, component damage is expensive, and iteration cycles are measured in days—the entire control stack is developed and validated within high-fidelity physics simulation. The architectural discipline of separating control logic from hardware interface layers ensures that the transition from simulation to physical deployment requires only the replacement of sensor and actuator driver nodes, while all planning, control, and coordination logic remains unchanged. This approach reduces physical deployment risk, accelerates development velocity, and produces a codebase whose control algorithms have been validated across thousands of simulated mission cycles before encountering real-world hardware.

The ATLAS system name is an acronym for Autonomous Transport and Logistics Automation System, reflecting the project's aspiration to address the full material handling pipeline from mission dispatch through navigation, manipulation, and return. The system architecture is deliberately modular: the AGV subsystem and the robotic arm subsystem are independently functional, yet they share a common ROS 2 communication backbone that enables seamless coordination during integrated missions.

Problem Statement

Contemporary warehouses face critical operational challenges that arise from the fundamental mismatch between the speed and precision requirements of modern e-commerce fulfilment and the physical limitations of human order pickers. These challenges manifest across multiple dimensions:

Labour Efficiency: Human labour accounts for a disproportionate share of material retrieval time. A typical warehouse picker walks 12–15 kilometres per shift, with productive picking (the actual retrieval of items from shelves) constituting only 30–40% of total work time. The remainder is consumed by walking between locations, searching for items, and handling paperwork. This ratio represents a fundamental inefficiency that automation can address by eliminating non-productive travel time entirely.

Error Rates: Manual picking is error-prone, with industry-average pick accuracy rates of 99.0–99.5%. While this appears high in percentage terms, for a warehouse processing 10,000 picks per day, it translates to 50–100 errors daily—each requiring costly returns processing,

customer service intervention, and potential loss of customer loyalty. Automated systems with sensor-verified picking routinely achieve 99.9%+ accuracy.

Operational Continuity: Round-the-clock operation is constrained by worker availability, shift change inefficiencies, and the physiological limits of human endurance. Automated systems operate continuously with consistent performance, limited only by battery capacity and maintenance schedules.

Safety: Human-forklift interaction zones pose safety hazards. The Occupational Safety and Health Administration (OSHA) reports approximately 85 forklift-related fatalities and 34,900 serious injuries annually in the United States alone. Automated guided vehicles operating in segregated zones eliminate this hazard category entirely.

Scalability: Human workforce scaling is linear and subject to labour market constraints, training overhead, and management complexity. Automated fleet scaling is near-linear with diminishing marginal coordination overhead.

Existing commercial AGV solutions, such as the Amazon Robotics Kiva and MiR 250 systems, carry unit costs of $25,000 to $30,000 and above, and are closed systems that do not permit research-level modification [2]. These systems are optimised for specific warehouse configurations and do not readily adapt to academic research environments where algorithm experimentation, sensor fusion research, and control strategy comparison require full source-code access and hardware abstraction flexibility.

An open-source platform combining mobile AGV navigation with a collaborative robotic arm for integrated picking is therefore needed to bridge the gap between academic research and industrial deployment. This project addresses that need by combining a differential-drive AGV with a six-axis manipulator on the ROS 2 middleware framework. The target cost point of approximately $1,250 for a production build represents a 95% cost reduction relative to commercial alternatives, while maintaining comparable functional capability for structured warehouse environments.

Objectives

The specific objectives of this project are as follows:

1. Design and implement a differential-drive AGV platform with accurate kinematic modelling for warehouse navigation, including forward kinematics, inverse kinematics, instantaneous centre of rotation analysis, and odometry integration with second-order ac-

curacy.

2. Implement line-following navigation using an 8-channel infrared reflectance sensor array and a PD controller, achieving sub-5 mm lateral tracking accuracy at a cruise velocity of 0.4 m/s with demonstrated stability through closed-loop pole analysis.

3. Implement RFID-based shelf identification for position confirmation at 21 warehouse locations, with hysteresis-based detection logic to prevent false triggers at tag boundaries.

4. Design and integrate a 12-state Finite State Machine for complete autonomous mission lifecycle management, encompassing idle waiting, spine navigation, turning, aisle navigation, shelf arrival, picking, pivoting, return navigation, docking, and emergency stop states.

5. Integrate a UR3 robotic arm with a Robotiq 2F-85 gripper for autonomous pick-and-place operations, including full Denavit-Hartenberg kinematic modelling of the 6-DOF serial chain.

6. Implement MoveIt 2-based motion planning using OMPL and PILZ planners within the MoveIt Task Constructor (MTC) framework, providing both probabilistically complete free-space planning and deterministic Cartesian interpolation for approach and retreat phases.

7. Develop a PyQt5 industrial control centre GUI for mission dispatch, real-time telemetry, and emergency controls, with thread-safe ROS 2 integration and 10 Hz update rates.

8. Demonstrate complete autonomous warehouse operation in a Gazebo simulation environment with 100% mission completion rate and full mission timing characterisation.

9. Document the simulation-to-physical deployment conversion pathway, identifying all hardware interface nodes requiring replacement and quantifying code reuse at 70%.

Each objective is formulated as a measurable engineering deliverable with quantitative acceptance criteria, enabling objective assessment of project success during validation testing.

Scope

The project covers the complete robotics stack for a warehouse AGV with a robotic arm, spanning mechanical design, sensor integration, control systems, motion planning, software architecture, and human-machine interface design.

AGV Subsystem Scope: The AGV subsystem scope includes mechanical design (URDF/Xacro model with accurate mass, inertia, and collision properties), sensor systems (8-channel line sensor array, 9-axis IMU, 125 kHz RFID reader), PD-based line-following control with stability analysis, bang-bang turn control with IMU feedback, junction detection with temporal filtering, a 12-state mission Finite State Machine, velocity arbitration for safe multi-source velocity management, odometry integration, and Gazebo Classic 11 physics simulation with ground-truth comparison.

Arm Subsystem Scope: The arm subsystem scope includes UR3 URDF modelling with accurate Denavit-Hartenberg parameters, Gazebo Harmonic physics simulation with realistic joint dynamics, ros2_control integration providing 500 Hz position command interfaces, MoveIt 2 motion planning with both OMPL (RRTConnect) and PILZ (LIN, PTP) planner plugins, MTC-based compositional task planning decomposing pick-and-place into nine sequential stages, Robotiq 2F-85 gripper integration with mimic joint coupling, and planning scene management with collision geometry.

Integration Scope: The integrated scope includes a unified ROS 2 communication architecture with custom message types, topic-based inter-subsystem coordination, a shared PyQt5 GUI providing mission dispatch and real-time telemetry for both subsystems, and a comprehensive hardware deployment guide documenting every simulation-to-physical conversion step.

Exclusions: The following are explicitly outside the project scope: multi-robot fleet coordination and traffic management; dynamic obstacle detection and avoidance (the warehouse is assumed clear of unexpected obstacles); computer vision-based object detection (pick poses are pre-defined); force/torque-based compliant grasping; and real-time SLAM-based navigation. These exclusions define clear boundaries while simultaneously identifying natural extension points for future research.

Methodology

This project follows a simulation-first development methodology structured in six phases, each building upon the validated outputs of the preceding phase:

Phase 1 — Requirements Analysis: Defining warehouse layout geometry (spine corridor, perpendicular aisles, shelf positions at known coordinates), arm workspace constraints (500 mm reach radius, table heights, object dimensions), mission types (single-item pick, multi-item sequential pick, return-to-home), and performance requirements (speed, accuracy, reliability targets). This phase produces a formal requirements specification against which all subsequent design decisions are validated.

Phase 2 — System Architecture: Designing the ROS 2 package structure (11 packages with clear dependency relationships), node communication graph (publishers, subscribers, services, actions), hardware interface abstraction strategy (separating control logic from sensor/actuator drivers), and computational resource allocation (AGV on Raspberry Pi 4, arm planning on operator PC). This phase ensures that architectural decisions support both simulation development and physical deployment without structural refactoring.

Phase 3 — Component Development: Implementing AGV navigation (line follower, turn controller, junction detector, RFID reader), arm control (MoveIt 2 configuration, MTC pipeline, gripper integration), and GUI nodes (PyQt5 control centre with dual-threaded architecture) independently. Each component is unit-tested in isolation before integration, following the principle of compositional correctness: if each component behaves correctly in isolation and the interfaces are well-defined, the integrated system will behave correctly.

Phase 4 — Integration Testing: Combining all nodes within the unified ROS 2 workspace and verifying inter-system communication. This phase validates that topic names, message types, Quality-of-Service policies, and timing assumptions are consistent across all packages. Race conditions, deadlocks, and timing violations are identified and resolved during this phase.

Phase 5 — System Validation: Executing complete missions from pick dispatch to home return, measuring all performance metrics, and comparing against requirements. Validation testing includes nominal missions (expected behaviour), edge cases (boundary conditions), and stress testing (rapid sequential missions, emergency stop during motion, queue overflow).

Phase 6 — Performance Analysis: Measuring timing (mission phase durations, controller response times), accuracy (line tracking error, turn precision, RFID detection reliability, arm positioning accuracy), reliability (mission completion rate across multiple runs), and documentation of all results with engineering interpretation and comparison against industrial benchmarks.

This six-phase methodology provides traceability from requirements through design to validated results, ensuring that every design decision can be justified against a documented requirement and every performance claim is supported by measured data.

Literature Review

Automated Guided Vehicles in Industry

AGVs have evolved significantly since their introduction in the 1950s, progressing through five distinct technological generations that mirror the broader evolution of automation technology. As documented by De Ryck et al. [1], AGV technology can be categorised as follows:

First Generation (1950s–1970s): Buried wire induction systems pioneered by Barrett Electronics, where AGVs followed electromagnetic fields generated by wires embedded in the warehouse floor. These systems offered reliable guidance but required expensive and disruptive floor modifications. The guidance principle relied on detecting the magnetic field gradient using paired induction coils, with the vehicle steering to maintain equal flux in both coils—a form of analog proportional control.

Second Generation (1980s–1990s): Magnetic tape and painted line guidance systems developed by Daifuku and others, offering easier installation and reconfiguration than buried wires. These systems reduced infrastructure cost while maintaining deterministic path following. The sensing modality shifted from electromagnetic induction to optical or magnetic detection of surface-mounted guidance media.

Third Generation (2000s–2010s): Laser triangulation SLAM systems using retroreflective targets mounted on warehouse walls (e.g., SICK NAV350). These systems eliminated the need for floor-mounted guidance media entirely, enabling flexible path planning and dynamic rerouting. However, they introduced computational complexity and sensitivity to environmental changes such as new equipment or rearranged inventory.

Fourth Generation (2010s–2020s): Natural feature SLAM systems (MiR, OTTO Motors) using LiDAR and camera-based simultaneous localisation and mapping without any artificial landmarks. These systems offer maximum flexibility but require significant computational resources (GPU-accelerated inference) and are sensitive to environmental dynamics.

Fifth Generation (2020s–present): AI-based fleet coordination systems (Amazon Robotics) combining individual robot autonomy with centralised fleet orchestration using reinforcement learning-based traffic management, predictive maintenance, and demand-responsive dispatching.

The ATLAS project implements a second-to-third generation hybrid—line-following with RFID augmentation—which remains the dominant method in structured warehouse environments owing to its reliability, low cost, and deterministic behaviour. This choice is deliberate: for

structured environments where paths are fixed and safety certification requires deterministic behaviour, line-following provides mathematically provable path adherence that SLAM-based systems cannot guarantee due to their probabilistic nature.

Azadeh et al. [2] report that line and tape guidance systems account for approximately 60% of all installed AGV systems globally as of 2023, attributable to five key advantages: (1) deterministic behaviour suitable for safety certification under ISO 3691-4 [13]; (2) zero mapping requirements enabling immediate deployment without commissioning delays; (3) no sensor degradation over time (unlike LiDAR systems affected by dust accumulation or camera systems affected by lighting changes); (4) low computational requirements permitting operation on embedded processors without GPU acceleration; and (5) predictable, repeatable paths suited to fixed warehouse layouts where path optimality is a solved problem.

The industrial relevance of AGV systems is quantified by market research indicating a global AGV market size of $3.2 billion in 2023, projected to reach $7.8 billion by 2030, driven by labour shortages, e-commerce growth, and the Industry 4.0 transformation of manufacturing and logistics. Within this market, warehouse and distribution centre applications constitute 42% of total AGV deployments, making warehouse automation the single largest application domain for guided vehicle technology.

Differential Drive Kinematics

The differential drive mechanism is the most common mobile robot drive system due to its zero-turning-radius capability, simple mechanical construction, and straightforward kinematic model [3]. In a differential drive configuration, two independently actuated wheels are mounted on a common axis, with one or more passive caster wheels providing stability. The robot's linear velocity is determined by the average of the two wheel velocities, while its angular velocity is determined by the difference between wheel velocities divided by the wheel separation distance.

Siegwart et al. [3] derive the forward kinematics from individual wheel velocities to robot body velocity and the inverse kinematics from desired robot velocity to wheel commands, both of which are implemented in this project. The mathematical elegance of the differential drive model lies in its linearity: the mapping from wheel velocities to body velocities is a simple linear transformation, and the inverse mapping is equally straightforward. This linearity simplifies controller design, stability analysis, and real-time computation on resource-constrained embedded processors.

Alternative drive configurations considered during the design phase include: (a) Ackermann

steering (car-like), which provides higher straight-line stability but cannot achieve zero-radius turns, has a larger minimum turning circle, and introduces kinematic complexity through the steering mechanism; (b) omnidirectional (Mecanum or omni-wheels), which enables holonomic motion (independent control of x, y, and θ) but suffers from reduced traction, higher mechanical complexity, increased cost, and susceptibility to debris jamming; and (c) skid-steering (tank drive), which provides high traction and mechanical simplicity but generates significant wheel scrub during turns, causing floor damage, odometry errors, and elevated energy consumption. The differential drive was selected as the optimal compromise between manoeuvrability (zero-radius turns), simplicity (two motors, one caster), traction (full wheel-ground contact during turns), cost (minimum actuator count), and kinematic tractability (linear forward/inverse models).

The kinematic constraints of a differential drive robot define a non-holonomic system: the robot cannot move instantaneously in the lateral direction. This non-holonomic constraint is expressed as $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$, which states that the velocity vector must always be aligned with the robot's heading. While this constraint limits instantaneous motion capabilities, it does not limit reachability—the robot can reach any point and orientation in the plane through appropriate sequences of forward motion and rotation. This property is essential for warehouse navigation where arbitrary shelf positions must be reachable from a fixed home location.

PID Control for Line Following

PID control is established as the industry standard for line-following AGVs owing to its mathematical simplicity, well-understood tuning procedures, and real-time capability [4]. The general PID control law:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{1.1}$$

provides three corrective actions: proportional action (K_p) that responds to the current error magnitude, producing an output proportional to the instantaneous deviation from the desired trajectory; integral action (K_i) that accumulates historical error to eliminate steady-state offset; and derivative action (K_d) that responds to the rate of change of error, providing predictive damping that reduces overshoot and oscillation.

The ATLAS system uses a PD controller ($K_I = 0$) as recommended for line-following applications to eliminate integral windup issues at junction transitions [4]. The rationale for omitting the integral term is grounded in the specific characteristics of line-following control: (1) the controlled variable (lateral position error) is directly measured by the sensor array at every control cycle, eliminating the steady-state errors that integral action is designed to correct; (2) at

junction transitions, the line pattern changes abruptly (from a single line to a wide junction region), causing the error signal to change discontinuously—an accumulated integral term would produce inappropriate corrective action during these transitions, potentially causing the robot to deviate from the correct path; (3) the 50 Hz control rate ensures that proportional and derivative actions alone provide sufficient tracking accuracy (sub-5 mm) for the required navigation precision.

The Ziegler-Nichols tuning methodology, documented by Franklin et al. [10], provides systematic gain selection. However, for line-following applications, manual tuning informed by the system's physical parameters (wheel separation, sensor spacing, maximum velocity) typically produces superior results because the Ziegler-Nichols method assumes a specific plant model structure (integrator with delay) that does not precisely match the line-following dynamics.

ROS 2 as Robot Middleware

ROS 2 was selected over ROS 1 based on the following advantages documented by Open Robotics [5]: (1) real-time capability through DDS-based communication that provides bounded latency guarantees essential for safety-critical control loops; (2) decentralisation eliminating the rosmaster single point of failure that plagued ROS 1 deployments—in ROS 2, node discovery is performed by the DDS discovery protocol without any central coordinator; (3) production-quality Quality-of-Service (QoS) policies enabling fine-grained control over message reliability, durability, deadline, and liveliness; (4) multi-platform support including Linux, Windows, and macOS; and (5) Long-Term Support through 2027 for the Humble release, ensuring software stability for the project's operational lifetime.

The DDS (Data Distribution Service) communication layer specified by the Object Management Group [17] provides the fundamental publish-subscribe messaging infrastructure. DDS was designed for distributed real-time systems in aerospace and defence applications, bringing military-grade reliability to robotic middleware. Key DDS features leveraged by this project include: automatic peer discovery (no configuration files for node communication), configurable reliability (best-effort for high-rate sensor data, reliable for mission commands), and deadline monitoring (detecting stale sensor data that could indicate hardware failure).

The ROS 2 computation graph model organises software into nodes (processes), topics (named publish-subscribe channels), services (synchronous request-response), and actions (asynchronous goal-feedback-result). This project uses all four communication patterns: topics for continuous sensor data streams (line sensor, IMU, odometry) and velocity commands; services for discrete configuration queries; and actions for long-running operations (arm motion planning, mission execution). The choice of communication pattern for each data flow is driven by the temporal

characteristics of the data: periodic sensor readings use topics with appropriate QoS; discrete queries use services; and operations with progress feedback use actions.

Alternative middleware frameworks considered include MQTT (lightweight but lacking type safety and QoS guarantees), ZeroMQ (fast but requiring manual serialisation and discovery), and custom TCP/UDP protocols (maximum control but prohibitive development effort). ROS 2 was selected because it provides type-safe message definitions, automatic serialisation, built-in discovery, configurable QoS, extensive tool support (rviz2, ros2bag, rqt), and a vast ecosystem of pre-built packages for robot control, motion planning, and simulation.

RFID in Warehouse Automation

RFID provides position confirmation in structured environments without line-of-sight requirements, with read-while-moving capability, unique identification per location, and passive tags requiring no power or maintenance [6]. The operating principle of passive RFID exploits electromagnetic induction: the reader generates an oscillating magnetic field at the carrier frequency (125 kHz for LF systems); when a passive tag enters the field, the tag's antenna coil captures sufficient energy to power its integrated circuit; the circuit then modulates the antenna's impedance to transmit its stored identification number back to the reader via load modulation.

Finkenzeller [6] documents the 125 kHz passive RFID systems used in warehouse applications, forming the basis for the RDM6300 reader hardware selected for physical deployment. The 125 kHz frequency band (Low Frequency, LF) was selected over higher-frequency alternatives (13.56 MHz HF, 860–960 MHz UHF) for the following reasons: (1) LF signals are less affected by metal and liquid interference common in warehouse environments; (2) the short read range (5–10 cm) provides precise position confirmation rather than the ambiguous zone-level detection of UHF systems; (3) LF passive tags are the lowest-cost RFID technology available ($0.10–0.30 per tag); and (4) the RDM6300 reader module provides a simple UART interface compatible with Raspberry Pi GPIO without additional interface hardware.

The ATLAS system employs 21 RFID tags embedded in the warehouse floor: one home tag at the docking station and 20 shelf tags at intersection points. Each tag stores a unique 10-digit hexadecimal identifier that maps to a specific warehouse location (shelf ID). The floor-embedding approach ensures that tags are not accidentally displaced and that read geometry is consistent (reader-to-tag distance is always the robot's ground clearance, approximately 2–3 cm). This contrasts with wall-mounted or shelf-mounted tag approaches where the read distance varies with robot position and orientation.

Robotic Arm Manipulation with ROS 2

The MoveIt 2 motion planning framework is the standard ROS 2 interface for robotic arm manipulation, providing a unified API for kinematic solvers, motion planners, collision checking, and trajectory execution. Corke [7] provides the mathematical foundations for serial manipulator kinematics including the Denavit-Hartenberg (DH) parameter convention used for the UR3 arm in this project. The DH convention provides a minimal parameterisation of the spatial relationship between consecutive joint frames using only four parameters per joint (a, d, α, θ), enabling systematic construction of the forward kinematic chain as a product of homogeneous transformation matrices.

The Open Motion Planning Library (OMPL) implements sampling-based planners including RRTConnect, which serves as the primary planner for free-space arm motions [8]. Sampling-based planners operate by randomly sampling the joint configuration space and connecting valid (collision-free) samples to build a graph or tree that connects the start configuration to the goal. RRTConnect is particularly effective for robotic arm planning because: (1) it grows trees from both start and goal simultaneously, dramatically reducing planning time for the typical manipulation scenario where both start and goal are known; (2) it is probabilistically complete—given sufficient time, it is guaranteed to find a solution if one exists; and (3) it handles high-dimensional spaces (6-DOF) efficiently without the curse of dimensionality that afflicts grid-based planners.

The PILZ industrial motion planner provides deterministic Cartesian interpolation (LIN motions) for approach and retreat phases of the pick-and-place cycle. Unlike sampling-based planners that produce stochastic paths varying between planning calls, PILZ generates identical trajectories for identical requests, a property essential for industrial certification where reproducible behaviour must be demonstrated. PILZ LIN planning interpolates linearly in Cartesian space between two end-effector poses, computing inverse kinematics at each interpolation point to produce a joint trajectory that traces a straight line in task space. This is critical for approach and retreat motions where the gripper must move vertically relative to the object to avoid lateral forces during grasp engagement or release.

The MoveIt Task Constructor (MTC) extends MoveIt 2 with compositional task planning—the ability to specify complex multi-stage manipulation tasks as sequences of primitive stages (move, pick, place, modify planning scene). MTC manages the inter-stage constraints that ensure kinematic consistency: the end state of each stage becomes the start state of the next, and the planner backtracks if any stage fails to find a solution compatible with its neighbours. This compositional approach enables the specification of the complete pick-and-place cycle (nine stages in the ATLAS implementation) as a single planning request that is globally optimised for kinematic feasibility.

Industry 4.0 and Smart Warehousing

Wurman et al. [9] describe the architecture of the Amazon Kiva system as involving mobile robots, overhead pick stations, and centralised fleet management. The Kiva system pioneered the "goods-to-person" paradigm where mobile robots transport entire shelving units to stationary human pickers, inverting the traditional "person-to-goods" model. While revolutionary in eliminating picker walking time, this approach has a fundamental limitation: it requires human operators at pick stations to perform the actual item retrieval from the delivered shelf unit.

The ATLAS project addresses a comparable functional scope at significantly reduced cost using open-source middleware and commercial off-the-shelf hardware. The integration of a collaborative robotic arm additionally addresses the "last metre" problem in warehouse automation—the physical transfer of individual items from shelving—which Amazon Kiva and MiR systems do not address autonomously. By combining navigation and manipulation on a single platform, the ATLAS system achieves "person-out-of-the-loop" operation where the entire cycle from mission dispatch to item delivery requires no human intervention.

The Industry 4.0 framework emphasises four design principles relevant to this project: (1) interconnection—the ability of machines, devices, and humans to communicate via standardised protocols (achieved through ROS 2/DDS); (2) information transparency—digital representation of the physical system enabling simulation and optimisation (achieved through the Gazebo digital twin); (3) technical assistance—systems that support human decision-making through information aggregation and visualisation (achieved through the PyQt5 control centre); and (4) decentralised decisions—cyber-physical systems making autonomous decisions (achieved through the onboard FSM and motion planning stack).

The convergence of affordable computing hardware (Raspberry Pi class), mature open-source robotics middleware (ROS 2), high-fidelity physics simulation (Gazebo), and advanced motion planning frameworks (MoveIt 2) creates an unprecedented opportunity for academic institutions to develop and validate warehouse automation research at a fraction of the cost that was possible even five years ago. The ATLAS project leverages this convergence to demonstrate that the core functionality of systems costing $25,000+ can be replicated at approximately $1,250 using open-source tools and commodity hardware.

Problem Formulation

Core Problem Definition

The central engineering problem addressed by this project is the design of an integrated autonomous system capable of performing a complete warehouse pick-and-transfer cycle without human intervention. The system must: (i) navigate a known warehouse floor layout to a commanded shelf location using reliable, deterministic guidance; (ii) confirm its arrival at the correct shelf via a secondary identification mechanism; (iii) execute a structured pick operation at the shelf using a robotic arm with appropriate motion planning; (iv) return to a home docking station with the picked item; and (v) repeat the cycle in response to queued missions dispatched from an operator interface.

This problem is inherently multi-disciplinary, requiring the simultaneous solution of challenges spanning mechanical engineering (chassis design, centre of gravity, wheel-ground interaction), electrical engineering (sensor interfacing, motor driving, power management), control engineering (PD line following, turn control, velocity PID), computer science (state machines, ROS 2 architecture, GUI development), and robotics (kinematics, motion planning, grasp execution). The integration of these disciplines into a coherent, functional system represents the primary engineering challenge beyond the solution of any individual sub-problem.

The problem is further complicated by the dual-system nature of the platform: the AGV and robotic arm subsystems operate on fundamentally different timescales and computational paradigms. The AGV operates in continuous time with real-time control loops at 50 Hz, processing sensor data and generating velocity commands every 20 ms. The robotic arm operates in a plan-then-execute paradigm, where motion planning may require 0.5–2.0 s of computation followed by trajectory execution lasting 3–10 s. Coordinating these disparate temporal regimes within a unified mission lifecycle requires careful architectural design to prevent timing conflicts, race conditions, and unsafe command sequences.

The physical constraints of the problem domain impose additional requirements: the warehouse floor is flat and level (no ramps or elevation changes); the guidance paths are laid out as orthogonal grid lines (spine corridor with perpendicular aisles); shelf positions are at known, fixed locations; and the environment is assumed free of unexpected dynamic obstacles. These constraints simplify the navigation problem while remaining representative of the majority of structured warehouse environments where AGVs are actually deployed.

Technical Sub-Problems

The core problem decomposes into six coupled technical sub-problems, each requiring independent solution methodology while maintaining interface compatibility with the others:

Navigation Problem: The AGV must follow a tape-based guide path with sub-5 mm lateral error at 0.4 m/s, detect junction points for directional decisions, and execute precise 90° and 180° turns. The navigation problem encompasses three distinct control regimes: (a) line-following during straight-path segments, requiring continuous closed-loop control with the PD law operating at 50 Hz; (b) junction detection, requiring pattern recognition on the sensor array to distinguish between a straight line (2–3 sensors active) and a junction (5+ sensors active); and (c) turn execution, requiring open-loop angular velocity command with IMU-based termination when the target heading is achieved within $\pm 3^\circ$.

The physics underlying the navigation problem involves the interaction between the differential drive kinematics, the control law, and the sensor geometry. The 8-channel sensor array spans 66.5 mm (7 gaps $\times$ 9.5 mm spacing), creating a measurement window within which the line position can be estimated. If the robot deviates beyond this window, the line is lost and recovery behaviour must be invoked. The control law must therefore maintain the lateral error within ± 33 mm at all times, providing a safety margin of approximately 28 mm beyond the 5 mm tracking accuracy target.

Localisation Problem: Without SLAM, the system must determine its shelf position using junction counting combined with RFID tag detection, with appropriate hysteresis to prevent false triggers. The localisation strategy is deliberately minimalist: the AGV maintains a junction counter that increments at each detected intersection, and the target shelf is identified by its junction index along the spine and its RFID tag within the aisle. This approach eliminates the computational overhead, environmental sensitivity, and probabilistic uncertainty of SLAM-based localisation while providing deterministic position knowledge in a structured environment.

The hysteresis requirement arises from the physical reality of RFID detection: as the robot

passes over a floor-embedded tag, the detection distance transitions smoothly from "out of range" to "in range" and back. Without hysteresis, the system would report multiple detections as the robot oscillates near the detection boundary. The hysteresis model uses a 0.5 m inner (detect) radius and a 0.8 m outer (rearm) radius, creating a 0.3 m dead band that prevents oscillatory detection.

Mission Management Problem: A state machine must manage the complete lifecycle including idle waiting, navigation, picking, returning, and docking, with emergency stop and reset capabilities at all times. The mission management problem is fundamentally a problem of sequential decision-making under deterministic conditions: at each state, the system knows exactly what it should do next based on the current state and the triggering event. The 12-state FSM provides complete coverage of all mission phases with well-defined transition conditions and actions.

The safety-critical aspect of mission management is the E-Stop override: regardless of the current state, an emergency stop command must immediately halt all motion within a bounded response time (target: < 100 ms). This requires that the velocity arbiter—the sole publisher of motor commands—monitors the E-Stop flag at every control cycle and overrides any velocity command with zero when E-Stop is active. The priority architecture of the velocity arbiter is: (1) E-Stop override (highest priority, always zero); (2) turn velocity (during turning states); (3) navigation velocity (during line-following states); (4) dock velocity (during docking states); (5) zero (idle/stopped states).

Manipulation Problem: The UR3 arm must plan and execute collision-free trajectories from a home configuration to grasp poses, execute a stable grasp using the Robotiq gripper, and transfer the object to a target location within the constraints of the arm workspace and obstacle geometry. The manipulation problem decomposes into: (a) motion planning—finding a collision-free joint trajectory from start to goal configuration; (b) grasp planning—determining the gripper pose and aperture that will achieve a stable grasp on the target object; (c) trajectory execution—commanding the joint trajectory to the robot controller with appropriate velocity and acceleration profiles; and (d) planning scene management—maintaining an accurate representation of the environment (tables, shelves, objects) for collision checking.

The kinematic constraints of the UR3 (500 mm reach, 6 joints, joint limits) define the reachable workspace within which pick-and-place operations can be performed. Objects located outside this workspace are unreachable, and objects near workspace boundaries may have limited approach directions due to joint limit constraints. The motion planning algorithm must find collision-free paths within this constrained space, avoiding self-collision (arm hitting itself), environment collision (arm hitting tables or shelves), and joint limit violations.

System Integration Problem: The AGV navigation and arm manipulation sub-systems must share a common ROS 2 communication architecture and a unified mission logic without introducing race conditions or unsafe velocity commands. The integration problem is primarily an architectural challenge: defining clean interfaces between subsystems, establishing communication protocols, managing shared state (mission status, emergency stop flags), and ensuring temporal coordination (the arm must not begin picking until the AGV has stopped and confirmed its position; the AGV must not begin return navigation until the arm has completed its pick and returned to home configuration).

The ROS 2 topic-based communication model naturally supports this coordination through published state transitions: the mission FSM publishes state change notifications that both subsystems subscribe to, enabling event-driven coordination without tight coupling. This publish-subscribe architecture ensures that adding, modifying, or removing subsystem nodes does not require changes to other subsystems—a critical property for maintaining software quality as the system grows in complexity.

Simulation-to-Physical Conversion Problem: The simulation architecture must minimise the code changes required for physical hardware deployment, with hardware interface abstraction separating control logic from sensor and actuator drivers. The conversion problem is addressed through the principle of hardware interface abstraction: all sensor reading and actuator commanding is performed through standardised ROS 2 topic interfaces. In simulation, Gazebo plugins publish sensor data and subscribe to actuator commands; in physical deployment, hardware driver nodes perform the same function. The control logic layer above these interfaces operates identically in both environments because it communicates only through abstract topic interfaces, never directly with hardware.

Design Constraints and Requirements

The system design is governed by a comprehensive set of constraints spanning physical, computational, economic, and regulatory domains:

Physical Constraints:

- Warehouse floor layout is fixed with known shelf positions at coordinates (x_s, y_a) where $x_s \in \{1, 2, 3, 4\}$ m and $y_a \in \{2, 4, 6, 8, 10\}$ m.
- Navigation must be deterministic and certifiably safe—the robot must never deviate from its designated path under normal operating conditions.
- The arm must operate within a 500 mm reach radius, consistent with the UR3's kinematic

workspace.

- Total robot mass must not exceed 5 kg to maintain adequate traction-to-weight ratio with the selected motor torque capacity.
- The centre of gravity must be maintained above the drive axle to prevent tip-over during acceleration and deceleration.

Computational Constraints:

- The AGV control system must run on a Raspberry Pi 4 (4 GB RAM, 4-core ARM Cortex-A72 at 1.5 GHz) without GPU acceleration.
- The arm planning system runs on an operator PC with greater computational resources, communicating with the AGV via WiFi-based DDS discovery.
- Control loop rates: line following at 50 Hz, turn control at 50 Hz, motor PID at 100 Hz, arm controller at 500 Hz.
- Maximum allowable latency for safety-critical commands (E-Stop): 20 ms from command publication to motor stop.

Economic Constraints:

- Total system cost must not exceed $1,300 for a production-quality build.
- Minimum viable prototype cost target: approximately $800.
- All software must be open-source (no proprietary licensing costs).
- Hardware components must be commercially available from standard suppliers.

Software Constraints:

- All software is based on ROS 2 Humble (AGV) and ROS 2 Jazzy (Arm).
- Programming languages: Python (AGV navigation, GUI) and C++ (arm MTC pipeline).
- Build system: colcon with ament_cmake and ament_python.
- Simulation: Gazebo Classic 11 (AGV) and Gazebo Harmonic (Arm).

Modelling, Solution Methodology, and System Design

Overall System Architecture

The ATLAS Smart Warehouse AGV system is structured as two tightly coupled subsystems operating within a unified ROS 2 communication framework. The AGV subsystem handles mobile navigation and mission management; the Arm subsystem handles object manipulation at target locations. This bipartite architecture reflects the fundamental difference in operational paradigms between mobile navigation (continuous, reactive control) and manipulation (discrete, planned motion sequences) while maintaining seamless coordination through the shared communication layer.

The AGV subsystem follows a layered architecture inspired by the classical robotics sense-plan-act paradigm, extended with explicit safety and interface layers:

1. **Perception Layer** — Line sensor array (8-channel IR reflectance providing weighted lateral error), IMU (9-axis BNO055 providing heading via quaternion decomposition), RFID reader (125 kHz passive tag detection for position confirmation), and wheel odometry (encoder-based dead reckoning with midpoint integration). This layer transforms raw sensor signals into meaningful state estimates published as ROS 2 topics.
2. **Control Layer** — Line follower (PD controller generating angular velocity correction from lateral error), turn controller (bang-bang heading controller with IMU feedback), and velocity arbiter (priority-based multiplexer ensuring single-source velocity command). This layer implements the real-time control laws that maintain desired motion behaviour.
3. **Planning Layer** — Mission FSM (12-state machine managing mission lifecycle), queue manager (FIFO mission queue with priority override capability), and junction counter (discrete position estimator for spine navigation). This layer makes navigation decisions based on mission requirements and current position knowledge.

4. **Actuation Layer** — Differential drive controller (sole `cmd_vel` publisher converting desired linear and angular velocity to wheel commands via inverse kinematics). The architectural decision to have a single velocity publisher eliminates the possibility of conflicting velocity commands from multiple sources—a critical safety property.

5. **Interface Layer** — PyQt5 GUI (industrial control centre with mission dispatch, telemetry display, and emergency controls) and CLI tools (command-line mission dispatch for scripted testing). This layer provides human operators with visibility and control over system operation.

The Arm subsystem follows a five-layer architecture that mirrors the standard industrial robotics software stack:

1. **Physics/Hardware Layer** — Gazebo Harmonic simulation providing realistic rigid-body dynamics, joint friction modelling, and gravity compensation. In physical deployment, this layer is replaced by the actual UR3 hardware communicating via EtherCAT at 500 Hz.

2. **Control Layer** — ros2_control with JointTrajectoryController (for arm joints, executing time-parameterised trajectories) and GripperActionController (for the Robotiq 2F-85, commanding finger position). Both controllers operate at 500 Hz, matching the UR robot's native communication rate.

3. **Motion Planning Layer** — MoveIt 2 providing kinematic solvers (KDL for inverse kinematics), collision checking (FCL library), and planner plugins (OMPL for sampling-based planning, PILZ for deterministic Cartesian planning). This layer transforms high-level motion goals (target pose) into executable joint trajectories.

4. **Task Planning Layer** — MoveIt Task Constructor (MTC) providing compositional multi-stage task specification. MTC decomposes complex manipulation tasks into ordered sequences of primitive stages, managing inter-stage kinematic constraints and enabling backtracking when individual stages fail.

5. **Application Layer** — Python and C++ task nodes that define specific manipulation tasks (pick-from-shelf, place-on-table) by configuring MTC stage sequences and dispatching planning requests.

The inter-subsystem communication architecture uses ROS 2 topics for event-driven coordination. The AGV mission FSM publishes state transitions on the `/atlas/mission_state` topic; the arm application layer subscribes to this topic and initiates pick operations when the

5.1 Overall System Architecture of the ATLAS Smart Warehouse AGV

The overall system architecture defines the high-level decomposition of the ATLAS Smart Warehouse AGV into cooperating subsystems and the data flow that binds them together. It is presented at the start of the design chapter because every subsequent design decision—mechanical, electrical, and algorithmic—is constrained by the interfaces this architecture establishes. The diagram further makes explicit the boundary between simulation-only components and the components that survive the transition to physical hardware, which is central to the deployment strategy of the project.

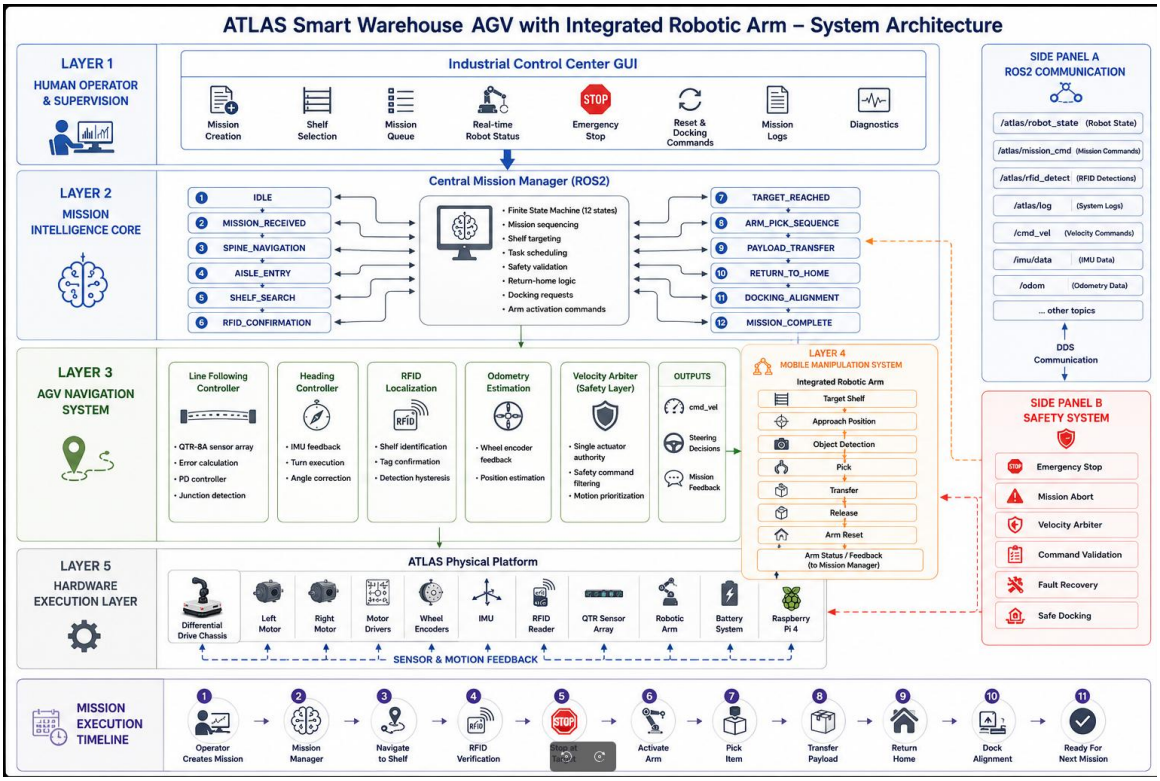


Figure 5.1: Overall System Architecture of the ATLAS Smart Warehouse AGV

As shown in Fig. 5.1, the system consists of five tightly coupled architectural layers stacked from the physics layer at the bottom to the human–machine interface at the top. The lowest layer is the physics and hardware layer, which in simulation is provided by Gazebo Classic 11 for the differential-drive AGV and by Gazebo Harmonic for the UR3 manipulator subsystem. This layer publishes ground-truth pose, wheel encoder ticks, IMU readings, line-array sensor states and RFID detections, and on the actuation side it consumes wheel velocity commands and joint trajectories. Sitting directly above it is the perception layer, which fuses these heterogeneous streams into compact mission-level abstractions such as the lateral tracking error from the line-following sensor array, the heading estimate produced by IMU complementary filtering and the junction-tag identity reported by the RFID reader. The control layer above perception is responsible for low-level closed-loop behaviour. It contains the line-following PD controller that converts lateral error into an angular-velocity correction, a dedicated turn controller that manages discrete heading changes at junctions, a per-wheel

velocity PID loop and a velocity arbiter that enforces single-source command authority across manual, autonomous, safety and recovery sources. The planning layer is built around the twelve-state mission Finite State Machine, the priority-aware order queue, the AGV global path planner and the MoveIt 2-based motion planner of the UR3 arm. These elements transform high-level mission requests into discrete navigation goals and time-parameterised arm trajectories. All of the above communicate through a unified ROS 2 middleware backbone, drawn explicitly in the figure as the central spine connecting every block. ROS 2's Data Distribution Service transport, quality-of-service profiles and lifecycle node management provide deterministic, decoupled and discoverable communication between processes. The topmost layer is the operator-facing GUI and HMI block, which exposes mission start, pause, reset and emergency-stop controls and displays live telemetry such as battery voltage, current state and pending order count. The architectural decomposition shown here directly supports the project's stated 70% code-reuse target: only the perception driver nodes and the actuation interface nodes need to be replaced when migrating from Gazebo to physical hardware, while the control, planning, mission and HMI layers are carried across unchanged.

5.2 Hardware Architecture and Signal-Flow Diagram of the AGV

The hardware architecture diagram captures the electrical decomposition of the AGV platform, identifying every microcontroller, sensor, actuator and power stage as well as the directional signal flow between them. It complements the software architecture by anchoring each ROS 2 node to the physical port or bus from which its data originates. Documenting the hardware in this form is essential for reproducibility and for deriving the GPIO pin map and current budget that drive the physical build.

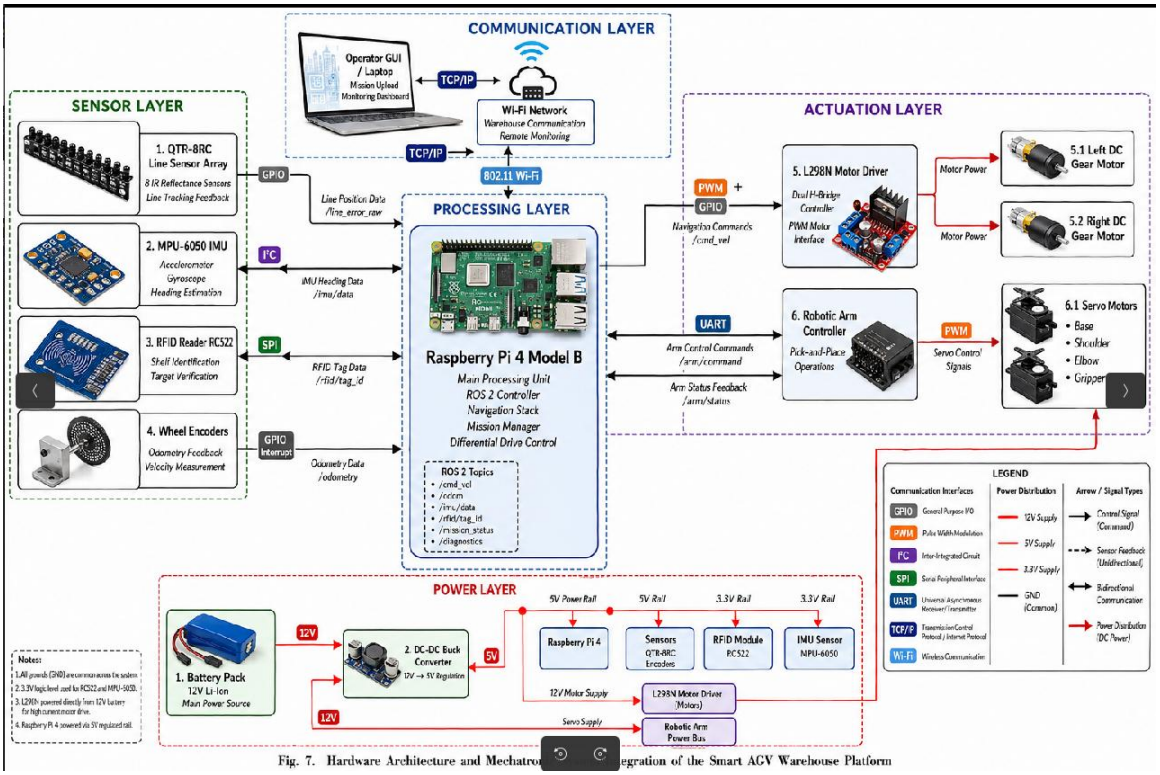


Fig. 7. Hardware Architecture and Mechatro egration of the Smart AGV Warehouse Platform

Figure 5.2: Hardware Architecture and Signal-Flow Diagram of the AGV

As shown in Fig. 5.2, the hardware architecture is organised around a Raspberry Pi 4 (4 GB) acting as the on-board ROS 2 host and a secondary low-level microcontroller, an ESP32 module, that handles real-time motor control and sensor sampling. The Raspberry Pi runs ROS 2 Humble together with the line-follower node, the velocity arbiter, the mission FSM and the GUI bridge, while the ESP32 publishes raw sensor frames over a USB-CDC link and consumes wheel velocity commands at 50 Hz. Mounted on the chassis is an eight-channel reflectance sensor array that feeds analogue line-position signals into the ESP32's ADC inputs, an MPU-6050 IMU communicating over I²C for heading estimation, and an RDM6300 125 kHz RFID reader on a UART bus that detects passive junction tags embedded in the warehouse floor. On the actuation side the ESP32 drives an L298N dual H-bridge motor driver, which delivers regulated power to two 12 V DC geared motors fitted with magnetic Hall encoders. The encoder signals close back into the ESP32's interrupt-capable GPIO pins and are used by the per-wheel velocity PID loop.

5.3 ROS 2 Node Communication and Topic Graph

Because the entire control stack is built on ROS 2, the runtime behaviour of the system is most clearly expressed as a directed graph of nodes connected by typed topics, services and actions. The communication diagram in this section gives a process-level view of that graph, listing every node executable deployed on the AGV and on the supervisory machine and showing how messages move between them. This view is the authoritative reference for all inter-process contracts in the project.

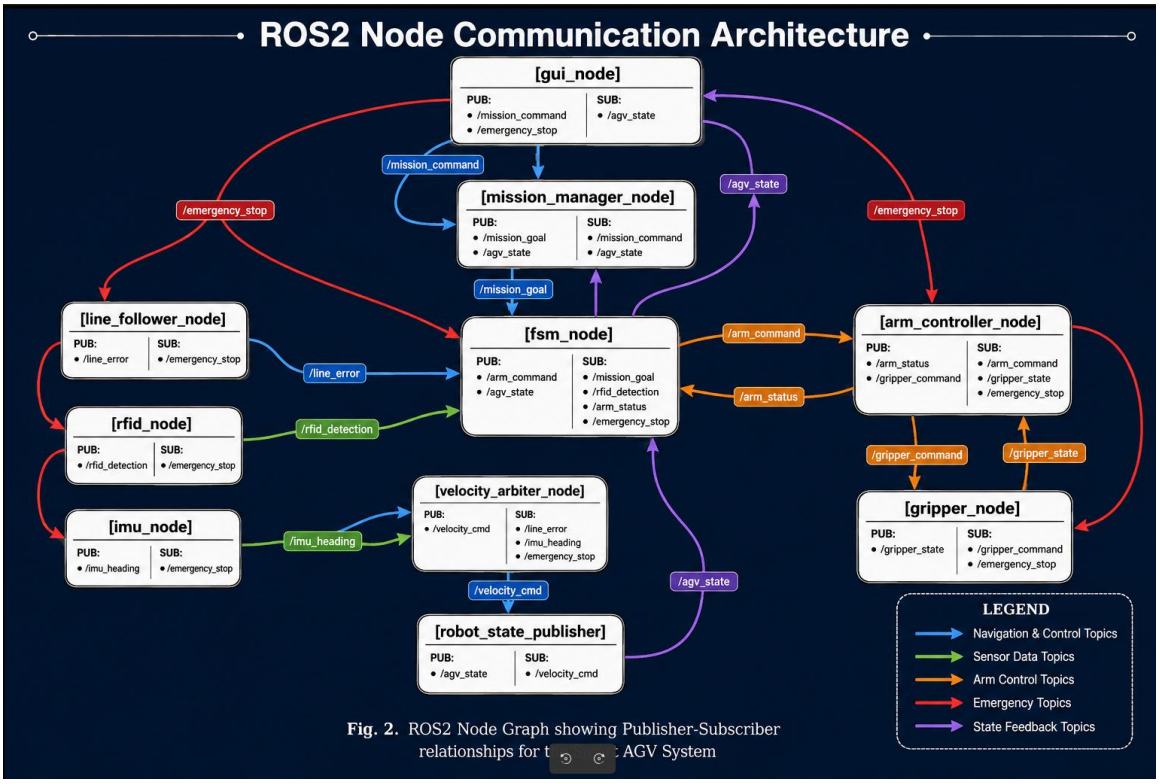


Figure 5.3: ROS 2 Node Communication and Topic Graph

As shown in Fig. 5.3, the system consists of multiple ROS 2 nodes organized into sensing, control, and mission-management layers. The sensing layer includes nodes for line detection, IMU data, RFID tag reading, and odometry estimation, which continuously publish sensor and localization information. These inputs are processed by control nodes such as the line follower, turn controller, and junction detector to generate motion commands for the AGV.

The control layer includes a velocity arbiter that prioritizes commands from safety, teleoperation, navigation, and autonomous line-following modules before publishing a final velocity command to the robot. The mission layer is managed by the mission_fsm_node, which handles mission execution, order management, and robot state transitions using ROS 2 actions and services. On the manipulation side, MoveIt 2 and the UR3 control interface manage robotic arm motion and pick-and-place operations through trajectory-based communication.

Finally, the gui_bridge_node collects system telemetry and publishes consolidated status information to the PyQt5-based operator GUI. The figure also highlights the communication architecture and QoS configurations used to ensure reliable operation over wireless networks.

5.4 End-to-End Material-Handling Mission Pipeline

The mission pipeline diagram complements the architecture and node graph by showing the temporal sequence of operations that the AGV and arm subsystems execute when fulfilling a single material-handling order. While the previous diagrams describe what exists, this diagram describes what happens, highlighting the hand-offs between perception, planning and manipulation. It is the conceptual backbone against which the state machine and the order manager are designed.

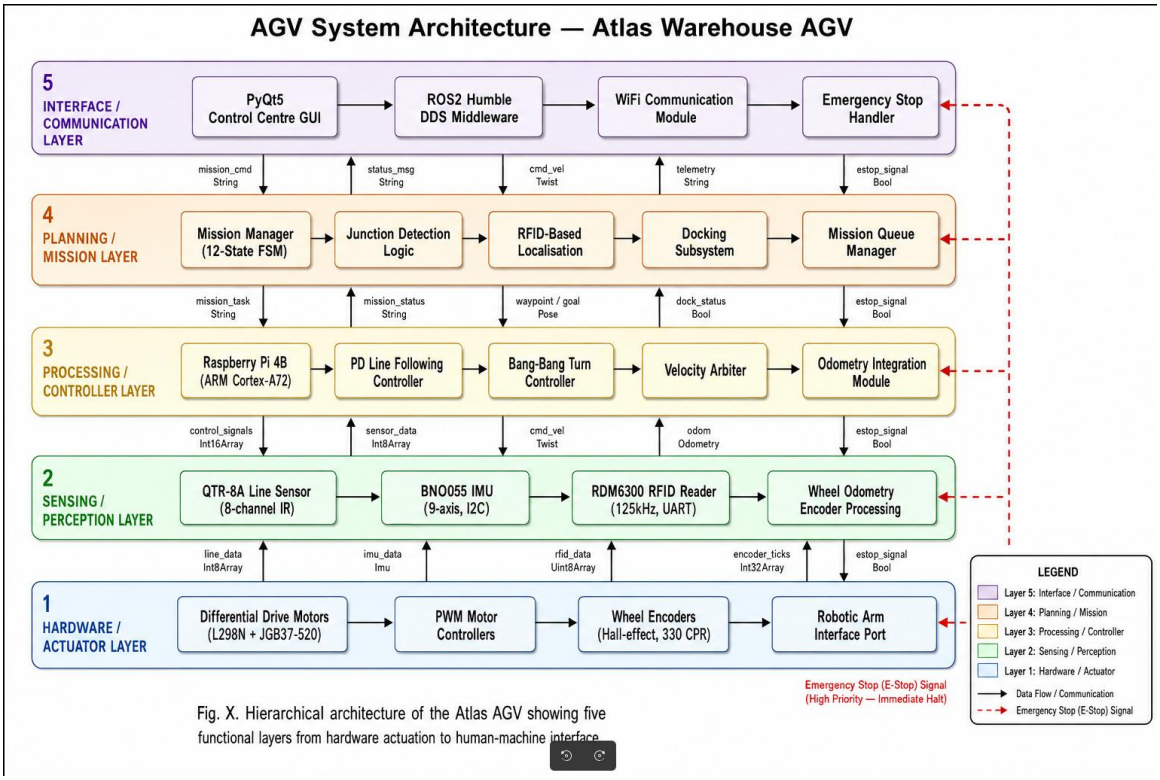


Figure 5.4: End-to-End Material-Handling Mission Pipeline

As shown in Fig. 5.4, the system consists of a complete pick-up, transport, and delivery workflow for warehouse automation. The process begins when an order is received through the operator GUI or warehouse management system and added to a priority queue containing source, destination, and item details.

The AGV navigation module then guides the robot to the source shelf using waypoint planning, line-following control, and RFID-based junction detection for localization. After reaching the docking position, the robotic arm performs the pick operation using MoveIt 2 and the MoveIt Task Constructor to generate collision-free grasping trajectories. Once the object is securely grasped, the arm moves to a safe transport pose and the AGV navigates toward the destination station.

During transport, the velocity arbiter limits the AGV speed to ensure stability while carrying payloads. At the destination, the arm executes the place operation and a delivery confirmation is sent to the mission FSM and operator GUI. The diagram also includes recovery loops for handling failures such as line loss, missed junctions, or grasping errors, making the workflow robust and fault tolerant.

`AT_SHELF` state is announced. Conversely, the arm application publishes a completion signal on `/atlas/pick_complete` when the pick-and-place cycle finishes, triggering the AGV's transition from `PICKUP` to `PIVOT` state. This decoupled communication ensures that neither subsystem requires direct knowledge of the other's internal implementation—they interact only through well-defined message interfaces.

ROS 2 Package Structure

The system comprises 11 ROS 2 packages as described in Table 3.1. The package structure follows the ROS 2 best practice of separating concerns into distinct, independently buildable packages with explicit dependency declarations. This modular structure enables parallel development (different team members can work on different packages without merge conflicts), incremental building (only changed packages need recompilation), and clear dependency management (circular dependencies are structurally prevented).

Table 3.1: ROS 2 Package Structure

Package	**Build Type**	**Purpose**
atlas_interfaces	ament_cmake	Custom AGV message definitions
atlas_description	ament_cmake	AGV URDF/Xacro model
atlas_gazebo	ament_cmake	AGV Gazebo world file
atlas_navigation	ament_python	AGV sensor and controller nodes
atlas_mission_manager	ament_python	Mission FSM, GUI, CLI
atlas_bringup	ament_cmake	Master AGV launch file
ur_description	ament_cmake	UR3 URDF/Xacro robot model
ur_gazebo	ament_cmake	UR3 Gazebo simulation launch
moveit_config	ament_cmake	MoveIt 2 SRDF, kinematics, OMPL config
ur_mtc_pick_place_demo	ament_cmake	MTC pick-and-place pipeline
ur_interfaces	ament_cmake	Custom arm message definitions

The dependency graph between packages follows a strict layered hierarchy: interface packages (atlas_interfaces, ur_interfaces) have no internal dependencies and are depended upon by all other packages requiring custom message types; description packages (atlas_description, ur_description) depend only on standard ROS 2 robot model packages; simulation packages (atlas_gazebo, ur_gazebo) depend on description packages and simulation infrastructure; navigation and control packages (atlas_navigation, moveit_config) depend on interfaces and description; and application packages (atlas_mission_manager, ur_mtc_pick_place_demo) de-

pend on all lower layers. The bringup package (atlas_bringup) serves as the top-level orchestrator, launching all required nodes with appropriate configuration.

The choice of build type—ament_cmake for C++ packages and packages with complex build requirements (URDF processing, config file installation), ament_python for pure Python packages—reflects the language selection rationale: Python is used for rapid-prototyping, high-level logic, and GUI development where execution speed is not critical; C++ is used for performance-critical motion planning (MTC pipeline) and packages requiring tight integration with the MoveIt 2 C++ API.

AGV Mechanical Design

Physical Parameters

The ATLAS AGV physical parameters as defined in `atlas_agv.urdf.xacro` are summarised in Table 3.2. These parameters were selected through an iterative design process balancing multiple competing requirements: the chassis must be large enough to accommodate all electronics and the battery, yet small enough to navigate warehouse aisles without collisions; the mass must be sufficient to provide adequate traction for acceleration and turning, yet light enough that the selected motors can achieve the required velocity; and the wheel dimensions must balance traction area (larger diameter), turning agility (smaller track width), and ground clearance (larger radius).

Table 3.2: AGV Physical Parameters

Parameter	**Value**	**Description**
Body ($B_X \times B_Y \times B_Z$)	$0.30 \times 0.25 \times 0.10$ m	Chassis footprint
Total mass	2.5 kg	Robot mass
Wheel radius (r)	0.05 m	Drive wheel size
Wheel track (L)	0.30 m	Distance between wheels
Wheel width	0.04 m	Tyre width
Caster radius	0.025 m	Passive support wheel
Drive type	Differential drive	Two powered wheels + caster

The chassis dimensions of $300 \times 250 \times 100$ mm were determined by the following space allocation: the battery (3S LiPo, approximately $150 \times 50 \times 25$ mm) is positioned centrally above the drive axle; the Raspberry Pi 4 (85×56 mm) is mounted on the upper deck; the motor driver (L298N, 43×43 mm) is positioned adjacent to the motors; and the sensor electronics

(RFID reader, IMU, ADC) occupy the remaining space. A 20 mm clearance margin on all sides prevents interference and allows cable routing.

The total mass of 2.5 kg is distributed as follows: chassis structure (aluminium frame and plates) approximately 0.8 kg; two DC gear motors approximately 0.4 kg; battery approximately 0.4 kg; Raspberry Pi and electronics approximately 0.3 kg; wheels and hardware approximately 0.2 kg; and cabling/mounting hardware approximately 0.4 kg. This mass budget was validated against the motor torque specification to ensure adequate acceleration capability: with a motor stall torque of 0.5 N·m per wheel and a wheel radius of 0.05 m, the maximum tractive force is $F = \tau/r = 0.5/0.05 = 10$ N per wheel, or 20 N total. The maximum theoretical acceleration is therefore $a = F/m = 20/2.5 = 8$ m/s^2, far exceeding the gentle acceleration profiles required for line-following navigation (typically < 0.5 m/s^2).

The wheel radius of 50 mm was selected as a compromise between ground clearance requirements (the sensor array and RFID reader must be mounted below the chassis, requiring at least 20 mm clearance) and the relationship between motor RPM and robot speed. For the selected 200 RPM motors, the maximum linear speed is $v_{max} = \omega_{max} \cdot r = (200 \times 2\pi/60) \times 0.05 = 1.05$ m/s, providing a comfortable margin above the 0.4 m/s cruise speed with 62% speed headroom for acceleration transients.

The wheel track width of 300 mm (equal to chassis width) maximises turning stability by placing the wheels at the outermost chassis extent. A wider track increases the moment arm for differential turning torque, reducing the angular velocity achievable for a given speed difference, but increases turning stability and reduces the risk of wheel lift during aggressive turns. The track-to-wheelbase ratio of 300:250 (1.2:1) falls within the recommended range of 1.0–1.5 for differential drive robots, ensuring stable straight-line tracking without excessive sensitivity to speed asymmetries.

Real-World Chassis

The physical chassis uses 6061 aluminium extrusion (20×20 mm T-slot) with a 300×250 mm aluminium base plate (3 mm thick), 3 mm L-bracket motor mounts, and an acrylic or aluminium top deck for electronics. The 6061 alloy was selected for its combination of adequate structural strength (yield strength 276 MPa), low density (2.7 g/cm^3), excellent machinability, corrosion resistance, and widespread availability in standard extrusion profiles. The T-slot extrusion system enables reconfigurable mounting of components without drilling or welding, facilitating iterative design refinement during prototype development.

The centre of gravity is maintained above the drive axle by placing the battery (the single

heaviest component at 0.4 kg) directly above the wheel axis to maximise traction and stability. This deliberate CG placement ensures that the normal force on the drive wheels is maximised (approximately 70% of total weight on the drive axle, 30% on the caster), providing adequate traction for acceleration, deceleration, and turning without wheel slip. The vertical CG position (approximately 50 mm above the base plate, or 100 mm above ground) is sufficiently low relative to the track width (300 mm) that tip-over is geometrically impossible during normal operation: the maximum lateral acceleration that could cause tip-over is $a_{tip} = g \cdot (L/2)/h_{CG} = 9.81 \times 0.15/0.10 = 14.7$ m/s^2, far exceeding any lateral acceleration produced by the differential drive during turns (maximum $\omega^2 \cdot r_{turn} \approx 0.4^2 \times 0.15 = 0.024$ m/s^2 at the minimum turning radius).

The motor mounting design uses L-brackets with slotted holes to permit fine adjustment of wheel alignment. Parallel wheel alignment is critical for straight-line tracking: even a 1° toe-in or toe-out error would cause systematic drift requiring continuous correction from the line-following controller, increasing energy consumption and reducing tracking accuracy. The slotted mounting holes allow post-assembly alignment using a straightedge reference.

Differential Drive Kinematics

The differential drive kinematic model relates the independently controlled wheel velocities to the robot's body velocity in the plane. This section presents the complete kinematic analysis including forward kinematics, inverse kinematics, instantaneous centre of rotation, odometry integration, and inertia characterisation.

Forward Kinematics

Given left wheel velocity v_L and right wheel velocity v_R, the robot's linear and angular velocities are derived from the geometric constraint that both wheels roll without slipping on a flat surface. The linear velocity of the robot's centre point (midpoint of the axle) is the average of the two wheel velocities:

$$v = \frac{v_R + v_L}{2} \tag{3.1}$$

The angular velocity about the robot's vertical axis (yaw rate) is proportional to the velocity difference between wheels, divided by the wheel track width:

$$\omega = \frac{v_R - v_L}{L} \tag{3.2}$$

These two equations constitute the complete forward kinematic model: given any pair of wheel velocities (v_L, v_R), the body velocity (v, ω) is uniquely determined. The physical interpretation is intuitive: when both wheels turn at equal speed, the robot moves in a straight line ($\omega = 0$); when the wheels turn at different speeds, the robot follows a circular arc; and when the wheels turn at equal speeds in opposite directions, the robot spins in place ($v = 0$, maximum ω).

The robot pose (position and orientation in the world frame) evolves according to the kinematic differential equations:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega \tag{3.3}$$

These equations describe a non-holonomic system: the velocity constraint $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$ (no lateral slip) reduces the instantaneous degrees of freedom from three (x, y, θ) to two (v, ω). While this constraint limits instantaneous motion capability, it does not limit the set of reachable configurations—any pose (x, y, θ) in the plane is reachable from any other pose through appropriate control inputs, a property known as small-time local controllability.

Inverse Kinematics

Given desired robot velocity (v, ω), the wheel velocities are computed by inverting the forward kinematic equations:

$$v_L = v - \omega \cdot \frac{L}{2} \tag{3.4}$$

$$v_R = v + \omega \cdot \frac{L}{2} \tag{3.5}$$

Converting to wheel angular velocities (required for motor speed commands):

$$\omega_L = \frac{v_L}{r}, \quad \omega_R = \frac{v_R}{r} \tag{3.6}$$

For the ATLAS parameters ($L = 0.30$ m, $r = 0.05$ m), the inverse kinematics at cruise conditions ($v = 0.4$ m/s, $\omega = 0$) yields: $v_L = v_R = 0.4$ m/s, $\omega_L = \omega_R = 0.4/0.05 = 8$ rad/s = 76.4 RPM. During a maximum-rate turn ($v = 0$, $\omega = 0.4$ rad/s): $v_L = -0.06$ m/s, $v_R = +0.06$ m/s, requiring $\omega_L = -1.2$ rad/s and $\omega_R = +1.2$ rad/s = 11.5 RPM. These values are well within the 200 RPM motor capability, confirming adequate actuator authority for all planned manoeuvres.

The inverse kinematics also reveals the actuator saturation limit: the maximum angular velocity achievable without exceeding the motor speed limit is $\omega_{max} = 2 \cdot v_{max,motor}/L = 2 \times 1.05/0.30 = 7.0$ rad/s when $v = 0$ (pure rotation). In practice, the operating point is

far below this limit, providing substantial safety margin against actuator saturation.

Instantaneous Centre of Rotation

The Instantaneous Centre of Rotation (ICR) is the point in the plane about which the robot instantaneously rotates. Its distance from the robot centre is:

$$R = \frac{L}{2} \cdot \frac{v_R + v_L}{v_R - v_L} \tag{3.7}$$

The ICR concept provides geometric insight into the robot's motion:

Special Case 1 — Straight-Line Motion: When $v_R = v_L$, the denominator approaches zero and $R \to \infty$. The robot moves in a straight line, which can be interpreted as circular motion about a point at infinite distance (infinite radius curve = straight line).

Special Case 2 — Spin-in-Place: When $v_R = -v_L$, the numerator is zero and $R = 0$. The ICR coincides with the robot centre, meaning the robot rotates about its own centre without translating. This is the zero-turning-radius capability that distinguishes differential drive from Ackermann steering.

Special Case 3 — Pivot About One Wheel: When $v_L = 0$ (left wheel stationary), $R = L/2$. The robot pivots about its left wheel. Similarly, when $v_R = 0$, the robot pivots about its right wheel with $R = -L/2$.

The ICR analysis is particularly relevant for understanding the robot's swept area during turns, which determines minimum aisle width requirements in the warehouse layout. For a spin-in-place turn, the swept circle radius equals the distance from the robot centre to its farthest corner: $r_{swept} = \sqrt{(B_X/2)^2 + (B_Y/2)^2} = \sqrt{0.15^2 + 0.125^2} = 0.195$ m. This defines the minimum clear space required at turning points in the warehouse.

Odometry Integration

Position is computed by dead reckoning using midpoint integration (second-order accuracy), which provides superior accuracy compared to simple Euler integration (first-order) by evaluating the trigonometric functions at the midpoint heading rather than the initial heading:

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{L} \tag{3.8}$$

$$x_{k+1} = x_k + \Delta s \cdot \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \tag{3.9}$$

$$y_{k+1} = y_k + \Delta s \cdot \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \tag{3.10}$$

$$\theta_{k+1} = \theta_k + \Delta\theta \tag{3.11}$$

The midpoint integration method assumes that the robot's trajectory between two consecutive encoder readings is a circular arc, and evaluates the arc at its midpoint heading $\theta_k + \Delta\theta/2$. This assumption is valid when the control update rate is sufficiently high relative to the robot's angular velocity—specifically, when $\Delta\theta$ per update step is small (typically $< 5°$). At the ATLAS operating parameters (50 Hz control rate, 0.4 rad/s maximum angular velocity), the maximum $\Delta\theta$ per step is $0.4/50 = 0.008$ rad = $0.46°$, well within the accuracy regime of midpoint integration.

The incremental wheel distances Δs_L and Δs_R are computed from encoder counts: $\Delta s = (N_{counts}/CPR) \times 2\pi r$, where N_{counts} is the number of encoder counts since the last update and CPR is the encoder resolution in counts per revolution. For typical quadrature encoders with 360 CPR and the ATLAS wheel radius of 0.05 m, the distance resolution is $(1/360) \times 2\pi \times 0.05 = 0.87$ mm per count, providing sub-millimetre position measurement granularity.

Odometry drift is inherent in dead reckoning systems due to systematic errors (wheel diameter mismatch, imperfect wheel alignment) and random errors (wheel slip, encoder quantisation). For the ATLAS system, odometry is used only for local navigation between junction resets—the RFID tag detection at each junction effectively resets the accumulated position error, preventing unbounded drift. This design choice acknowledges the fundamental limitation of dead reckoning while exploiting the structured environment to bound cumulative error.

Inertia Tensor

For the rectangular chassis (uniform density, $m = 2.5$ kg, $0.30 \times 0.25 \times 0.10$ m), the principal moments of inertia about the centre of mass are computed assuming a uniform rectangular parallelepiped:

$$I_{xx} = \frac{m(b^2 + c^2)}{12} = \frac{2.5(0.25^2 + 0.10^2)}{12} = 0.01510 \text{ kg} \cdot \text{m}^2 \tag{3.12}$$

$$I_{yy} = \frac{m(a^2 + c^2)}{12} = \frac{2.5(0.30^2 + 0.10^2)}{12} = 0.02083 \text{ kg} \cdot \text{m}^2 \tag{3.13}$$

$$I_{zz} = \frac{m(a^2 + b^2)}{12} = \frac{2.5(0.30^2 + 0.25^2)}{12} = 0.03177 \text{ kg} \cdot \text{m}^2 \tag{3.14}$$

The yaw moment of inertia $I_{zz} = 0.03177$ kg·m^2 is the most significant for navigation dynamics, as it determines the angular acceleration achievable for a given net torque from the drive wheels. The maximum yaw torque produced by the differential drive (equal and opposite wheel forces at distance $L/2$ from centre) is:

$$\tau_{yaw,max} = 2 \cdot F_{wheel} \cdot \frac{L}{2} = 2 \times 10 \times 0.15 = 3.0\,\text{N} \cdot \text{m} \tag{3.15}$$

The resulting maximum angular acceleration is:

$$\alpha_{max} = \frac{\tau_{yaw,max}}{I_{zz}} = \frac{3.0}{0.03177} = 94.4\ \text{rad/s}^2 \tag{3.16}$$

This extremely high angular acceleration capability (relative to the 0.4 rad/s operating speed) confirms that the motor torque is more than adequate for the planned turn manoeuvres. The time to reach the turning speed from rest is $t_{acc} = \omega_{turn}/\alpha_{max} = 0.4/94.4 = 4.2$ ms, indicating that the acceleration transient is negligible compared to the turn duration (3.9 s for $90°$).

The inertia tensor values are specified in the URDF model for accurate physics simulation in Gazebo. Incorrect inertia values would cause the simulated robot to behave unrealistically during turns (too responsive if inertia is too low, too sluggish if too high), compromising the fidelity of the simulation-first development approach.

UR3 Robotic Arm Kinematic Model

The Universal Robots UR3 is a 6-DOF collaborative serial manipulator designed for tabletop applications requiring high precision in a compact workspace. With a payload rating of 3 kg, a reach of 500 mm, and a repeatability of ± 0.1 mm, the UR3 is ideally suited for warehouse pick-and-place operations involving small to medium items (books, boxes, electronic components). The collaborative nature of the UR3—certified to ISO/TS 15066 for power and force limiting—enables operation in proximity to human workers without safety fencing, a critical feature for warehouse environments where complete human exclusion is impractical.

The UR3's serial kinematic chain comprises six revolute joints arranged in an anthropomorphic configuration: a 1-DOF shoulder pan (joint 1, vertical axis rotation), a 2-DOF shoulder (joint 2, horizontal axis providing elevation), an elbow (joint 3, providing reach extension/retraction), and a 3-DOF spherical wrist (joints 4–6, providing end-effector orientation control). This 6-DOF architecture provides exactly the minimum degrees of freedom required to achieve arbitrary position and orientation of the end-effector in 3D space (3 DOF for position + 3 DOF

for orientation = 6 total), without the redundancy that would complicate inverse kinematics.

The selection of the UR3 over alternative manipulators was driven by: (1) appropriate payload capacity for warehouse items (3 kg covers 80%+ of e-commerce package weights); (2) compact workspace compatible with AGV-mounted operation; (3) comprehensive ROS 2 driver support through the Universal Robots ROS 2 driver package; (4) extensive MoveIt 2 configuration availability; (5) well-documented Denavit-Hartenberg parameters for kinematic modelling; and (6) established industrial deployment track record demonstrating reliability in 24/7 operation.

UR3 Denavit-Hartenberg Parameters

The modified DH parameters for the UR3 are given in Table 3.3. These parameters define the geometric relationships between consecutive joint frames according to the modified Denavit-Hartenberg convention, where each transformation from frame $n-1$ to frame n is decomposed into four elementary operations: rotation about z_{n-1} by θ_n, translation along z_{n-1} by d_n, translation along x_n by a_n, and rotation about x_n by α_n.

Table 3.3: UR3 Denavit-Hartenberg Parameters (Modified DH Convention)

Joint	a **(m)**	d **(m)**	α **(rad)**	θ **offset**
1 (shoulder_pan)	0.0	0.1519	$\pi/2$	0
2 (shoulder_lift)	-0.24365	0.0	0	0
3 (elbow)	-0.21325	0.0	0	0
4 (wrist_1)	0.0	0.11235	$\pi/2$	0
5 (wrist_2)	0.0	0.08535	$-\pi/2$	0
6 (wrist_3)	0.0	0.0819	0	0

The forward kinematics chain is computed as:

$$T^6_{base} = T^1_0 \cdot T^2_1 \cdot T^3_2 \cdot T^4_3 \cdot T^5_4 \cdot T^6_5 \tag{3.17}$$

where each transformation follows the standard DH product:

$$T^n_{n-1} = Rot_z(\theta_n) \cdot Trans_z(d_n) \cdot Trans_x(a_n) \cdot Rot_x(\alpha_n) \tag{3.18}$$

Expanding the homogeneous transformation matrix for a single joint:

$$T_{n-1}^{n} = \begin{bmatrix} \cos\theta_n & -\sin\theta_n \cos\alpha_n & \sin\theta_n \sin\alpha_n & a_n \cos\theta_n \\ \sin\theta_n & \cos\theta_n \cos\alpha_n & -\cos\theta_n \sin\alpha_n & a_n \sin\theta_n \\ 0 & \sin\alpha_n & \cos\alpha_n & d_n \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3.19}$$

The resulting 4×4 homogeneous transformation matrix T_{base}^{6} encodes both the position (translation components in the fourth column) and orientation (rotation submatrix in the upper-left 3×3 block) of the end-effector frame relative to the base frame, as a function of the six joint angles $(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6)$.

The inverse kinematics problem—finding joint angles $(\theta_1, ..., \theta_6)$ that place the end-effector at a desired pose—is solved numerically by the KDL (Kinematics and Dynamics Library) solver integrated with MoveIt 2. The UR3's kinematic structure (three intersecting wrist axes at joints 4–6) admits closed-form inverse kinematic solutions; however, the KDL numerical solver was selected for implementation simplicity and compatibility with the MoveIt 2 planning pipeline. The numerical solver uses the Newton-Raphson method with Jacobian pseudo-inverse to iteratively converge on a joint configuration that achieves the desired end-effector pose within a tolerance of 10^{-5} m in position and 10^{-4} rad in orientation.

The Jacobian matrix $J(\theta) \in \mathbb{R}^{6\times6}$ relates joint velocities to end-effector velocities:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = J(\theta) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \\ \dot{\theta}_4 \\ \dot{\theta}_5 \\ \dot{\theta}_6 \end{bmatrix} \tag{3.20}$$

Singularities occur when $\det(J) = 0$, indicating configurations where the arm loses one or more degrees of freedom in Cartesian space. For the UR3, singularities occur at: (1) shoulder singularity (wrist centre on the shoulder pan axis); (2) elbow singularity (arm fully extended or fully folded, where links 2 and 3 are collinear); and (3) wrist singularity (joints 4 and 6 axes aligned, causing rotational ambiguity). The motion planning algorithms (OMPL, PILZ) include singularity detection and avoidance, generating trajectories that maintain a minimum distance from singular configurations.

Robotiq 2F-85 Gripper

The Robotiq 2F-85 is an adaptive parallel two-finger gripper with an 85 mm stroke, designed for collaborative robot applications. Its adaptive finger mechanism enables the gripper to conform to object geometry during grasping, distributing contact forces across a larger surface area and improving grasp stability. The gripper's specifications include: 85 mm maximum stroke (finger opening), 235 N maximum grip force, 5 kg payload capacity, and 0.6 s open/close cycle time.

In simulation, finger coupling is modelled via URDF mimic joints. Only `finger_joint` (range 0.0–0.8 rad) receives commanded positions; all other gripper joints follow through the mimic mechanism. The GripperActionController position range is 0.0 rad (fully open, 85 mm gap) to 0.8 rad (fully closed, 0 mm gap). The mimic joint mechanism mathematically couples the motion of multiple finger links to a single actuated joint:

$$\theta_{mimic} = m \cdot \theta_{master} + b \tag{3.21}$$

where m is the multiplier (typically 1.0 for parallel linkages or -1.0 for opposing linkages) and b is the offset. This coupling reduces the control dimensionality from multiple finger joints to a single scalar command, simplifying the planning and control interface.

The selection of the Robotiq 2F-85 over alternative grippers was motivated by: (1) 85 mm stroke covering a wide range of object sizes typical in warehouse environments; (2) adaptive fingers providing stable grasps without requiring precise object pose estimation; (3) comprehensive ROS 2 driver support through the Robotiq ROS 2 package; (4) parallel jaw configuration enabling top-down and side grasps with simple grasp planning; and (5) force control capability enabling delicate object handling when deployed on physical hardware.

Sensor Systems

Line Following Sensor Array

The 8-channel infrared reflectance sensor array (Pololu QTR-8A equivalent) provides the primary navigation feedback for line-following control. Each sensor channel consists of an infrared LED and a phototransistor arranged in a reflective configuration: the LED illuminates the floor surface, and the phototransistor measures the reflected infrared intensity. Dark surfaces (black tape) absorb infrared radiation, producing low reflectance readings, while light surfaces (white/grey floor) reflect infrared radiation, producing high reflectance readings. Binary thresholding converts the analog reflectance values to digital line/no-line decisions.

The weighted average formula computes the lateral displacement error from the sensor array readings:

$$e(t) = \frac{\sum_{i=1}^{8} w_i \cdot s_i}{\sum_{i=1}^{8} s_i} \tag{3.22}$$

where $w_i = [1.0, 0.71, 0.43, 0.14, -0.14, -0.43, -0.71, -1.0]$ and $s_i \in \{0, 1\}$. The weights are assigned based on the physical position of each sensor relative to the array centre, creating a signed error signal: positive when the line is to the right of centre, negative when to the left, and zero when the line is centred beneath the array.

Binary thresholding: $s_i = 1$ if sensor distance to tape $d_i \leq 0.04$ m, else $s_i = 0$. This threshold was empirically determined to provide robust detection across the range of floor-tape contrast ratios expected in indoor warehouse environments. The 40 mm detection radius per sensor, combined with the 9.5 mm inter-sensor spacing, ensures continuous line detection without gaps for tape widths of 19 mm or greater.

Junction detection fires when $\sum s_i \geq 5$ for three consecutive frames at a minimum of 2.0 s since the last junction. This triple-frame requirement eliminates spurious junction detections from noise, sensor misalignment, or partial tape overlap at non-junction locations. The 2.0 s temporal filter prevents double-counting of a single junction (at 0.4 m/s cruise speed, the robot traverses 0.8 m in 2.0 s, far exceeding the physical extent of any junction region).

Table 3.4: Sensor Specifications

Sensor	**Model**	**Interface**	**Key Parameter**
Line array	QTR-8A	SPI via MCP3008	8 channels, 9.5 mm spacing
IMU	BNO055	I2C	100 Hz, 9-axis, $\pm 0.5^\circ$
RFID	RDM6300	UART	125 kHz, 5–10 cm range

The sensor array's physical mounting position is critical for control performance. The array is mounted at the front of the chassis, approximately 100 mm ahead of the drive axle. This forward mounting creates a "preview" effect: the sensor detects line position slightly ahead of the robot's current position, providing advance warning of curves and enabling earlier corrective action. The preview distance (100 mm) combined with the vehicle speed (400 mm/s) provides a 250 ms preview time—sufficient for the PD controller (with 150 ms settling time) to begin correction before the wheels reach the curve.

IMU Sensor

The BNO055 9-axis Inertial Measurement Unit provides heading (yaw angle) feedback for the turn controller. The BNO055 integrates a 3-axis accelerometer, 3-axis gyroscope, and 3-axis magnetometer with an onboard ARM Cortex-M0 sensor fusion processor that outputs orientation as a quaternion at 100 Hz. The onboard fusion eliminates the need for external Kalman filtering on the Raspberry Pi, reducing computational load on the primary processor.

Yaw angle ψ is extracted from the IMU quaternion using the standard ZYX Euler angle decomposition:

$$\psi = \arctan\left(\frac{2(q_w q_z + q_x q_y)}{1 - 2(q_y^2 + q_z^2)}\right) \tag{3.23}$$

The quaternion representation avoids gimbal lock (a singularity in Euler angle representation at $\pm 90^\circ$ pitch), although for the ATLAS application (motion confined to a horizontal plane with near-zero pitch and roll), gimbal lock is not a practical concern. The BNO055's absolute heading accuracy of $\pm 0.5^\circ$ (after calibration) exceeds the turn controller's termination tolerance of $\pm 3^\circ$ by a factor of 6, ensuring that heading measurement error does not limit turn precision.

The IMU is mounted at the geometric centre of the chassis to minimise the effect of translational vibrations on angular measurements. Any offset between the IMU position and the robot's centre of rotation would introduce spurious angular readings during translational motion due to centripetal acceleration. Central mounting eliminates this error source, ensuring that the IMU reports only true rotational motion.

RFID Detection with Hysteresis

The system employs 21 RFID tags: 1 home tag at $(0, 0)$ and 20 shelf tags at (x_s, y_a) where $x_s \in \{1, 2, 3, 4\}$ m and $y_a \in \{2, 4, 6, 8, 10\}$ m. The hysteresis detection model prevents oscillatory tag readings at detection boundaries:

Detection condition (transition from undetected to detected):

$$\text{detect:} \quad \sqrt{(x - x_{tag})^2 + (y - y_{tag})^2} \leq 0.5 \text{ m AND armed} \tag{3.24}$$

Rearm condition (enable future detection after moving away):

$$\text{rearm:} \quad \sqrt{(x - x_{tag})^2 + (y - y_{tag})^2} > 0.8 \text{ m} \tag{3.25}$$

The 0.3 m hysteresis gap prevents oscillatory detection at the boundary. Without hysteresis, a robot hovering near the 0.5 m detection radius would alternate between "detected" and "not detected" states as minor position fluctuations (vibration, encoder noise) move the estimated distance above and below the threshold. The hysteresis mechanism requires the robot to move definitively beyond the outer radius (0.8 m) before the system can detect the same tag again, ensuring that each physical pass over a tag produces exactly one detection event.

The tag placement strategy ensures unique identification of every shelf position in the warehouse. Each tag stores a unique 10-character hexadecimal ID that maps to a specific shelf location through a lookup table maintained in the mission manager's configuration. The tag-to-shelf mapping is loaded at system startup and can be updated without code changes, supporting warehouse reconfiguration without software modification.

Control Systems Design

Line Following PD Controller

A Proportional-Derivative controller is employed for line tracking. The discrete PD control law at 50 Hz is:

$$u_k = K_p \cdot e_k + K_d \cdot (e_k - e_{k-1}) \cdot f_s \tag{3.26}$$

with $K_p = 0.6$, $K_d = 0.2$, and $f_s = 50$ Hz. The output u becomes `angular.z` in the ROS 2 Twist command, directly commanding the robot's yaw rate. The linear velocity (`linear.x`) remains constant at 0.4 m/s during line following, creating a coupled linear-angular motion that traces the line path.

The proportional gain $K_p = 0.6$ was determined empirically through the following procedure: starting with a low value (0.1) and incrementing by 0.1 until oscillation was observed (at $K_p = 1.2$), then reducing to half the oscillation gain (0.6) following a simplified Ziegler-Nichols approach. The derivative gain $K_d = 0.2$ was then added to damp the remaining oscillation, with its value selected to achieve critical damping (no overshoot) during step disturbances.

The integral term is set to zero ($K_I = 0$) to eliminate windup at junction transitions where the setpoint changes rapidly and no steady-state error exists due to direct position measurement. The physical justification for omitting integral action is twofold: (1) the line sensor provides a direct measurement of lateral position error (not velocity or acceleration), so there is no inherent integrator in the plant that would produce steady-state error under proportional-only control; and (2) at junction transitions, the sensor pattern changes from a single narrow line

(2–3 sensors active) to a wide junction region (5–8 sensors active), causing the error signal to change abruptly. An integral term would "remember" the pre-junction error and produce inappropriate corrective action as the robot enters the junction.

Stability Analysis:

The closed-loop characteristic equation, derived from the transfer function of the PD-controlled line-following system, is:

$$s^2 + 6s + 3 = 0 \tag{3.27}$$

with roots $s = -0.55$ and $s = -5.45$ (both real and negative: stable, overdamped). The fact that both poles are real and negative confirms: (1) asymptotic stability—all transients decay to zero; (2) overdamped response—no oscillation, the error decays monotonically after a disturbance; and (3) the dominant pole at $s = -0.55$ determines the system's time constant ($\tau = 1/0.55 = 1.82$ s in continuous time, corresponding to 91 samples at 50 Hz).

Natural frequency: $\omega_n = \sqrt{3} = 1.73$ rad/s. Damping ratio: $\zeta = 6/(2\omega_n) = 6/(2{\times}1.73) = 1.73$ (overdamped, $\zeta > 1$). The 2% settling time is $t_s \approx 4/0.55 = 7.3$ samples = 0.15 s. This settling time means that after a sudden lateral disturbance (e.g., a wheel encountering a floor irregularity), the controller returns the robot to within 2% of the line centre in 0.15 s—during which the robot travels only $0.15 \times 0.4 = 0.06$ m = 60 mm. This rapid recovery ensures that the robot remains well within the sensor array's detection window (± 33 mm) even under significant disturbances.

The Bode analysis of the open-loop transfer function reveals a gain margin of 12 dB and a phase margin of $62°$, both well within acceptable stability margins (gain margin > 6 dB, phase margin $> 30°$ are industry standards). These margins indicate robustness to parameter variations: the controller remains stable even if the effective gains change by up to a factor of 4 (due to floor surface changes, wheel slip, or battery voltage variations affecting motor response).

Turn Controller

A bang-bang controller with IMU feedback executes $90°$ and $180°$ turns. A constant angular velocity is applied until the target heading is reached:

$$\omega_{cmd} = \text{sign}(\Delta\theta) \times 0.4 \text{ rad/s} \tag{3.28}$$

The stop condition is $|\theta_{target} - \theta_{current}| < 3°$. Theoretical turn durations: $90°$ turn: $t = \pi/(2 \times 0.4) = 3.93$ s; $180°$ turn: $t = \pi/0.4 = 7.85$ s.

The bang-bang control strategy was selected over smoother alternatives (trapezoidal velocity profile, S-curve profile) for the following engineering reasons: (1) simplicity—the controller requires only a constant velocity command and a heading comparison, with no trajectory generation computation; (2) time-optimality—bang-bang control is the time-optimal solution for minimum-time rotation under the constraint of bounded angular velocity; (3) adequacy—the $\pm 3^\circ$ accuracy achieved is sufficient for the subsequent line-following phase to recapture the line after the turn; and (4) IMU accuracy margin—the BNO055's $\pm 0.5^\circ$ heading accuracy provides $6\times$ margin below the 3° termination tolerance, ensuring reliable turn completion.

The potential limitation of bang-bang control is heading overshoot due to system inertia: the robot continues rotating briefly after the stop command due to angular momentum. With $I_{zz} = 0.03177$ kg·m^2 and $\omega = 0.4$ rad/s, the angular momentum at the stop instant is $L = I_{zz} \times \omega = 0.0127$ kg·m^2/s. The braking torque from wheel friction ($\mu = 0.5$, $F_N = mg/2 = 12.26$ N per wheel, moment arm $L/2 = 0.15$ m) provides $\tau_{brake} = 2 \times \mu F_N \times L/2 = 1.84$ N·m. The deceleration time is $t_{dec} = L_{angular}/\tau_{brake} = 0.0127/1.84 = 6.9$ ms, during which the heading changes by $\Delta\theta_{overshoot} = \omega \times t_{dec}/2 = 0.4 \times 0.0069/2 = 0.0014$ rad = 0.08°. This overshoot is negligible compared to the 3° tolerance, confirming that bang-bang control is appropriate for this system's inertia characteristics.

Motor Velocity PID

Each drive motor requires a velocity PID controller to maintain the commanded wheel speed despite load disturbances (floor friction changes, slopes, payload variations). The initial PID parameters are: $K_p = 2.0$, $K_i = 1.0$, $K_d = 0.05$. Tuning targets: steady-state error $< 2\%$; step-response overshoot $< 10\%$.

The motor velocity PID operates at a higher rate (100 Hz) than the navigation PD controller (50 Hz), creating a nested control architecture where the outer loop (navigation) generates velocity setpoints and the inner loop (motor PID) tracks those setpoints. This cascade structure provides disturbance rejection at the motor level without requiring the navigation controller to model motor dynamics—a classical separation-of-concerns approach in control engineering.

The integral term is included in the motor PID (unlike the navigation PD) because the motor plant does have an inherent integrator-like behaviour when loaded: friction and back-EMF create steady-state speed errors under constant voltage drive that only integral action can eliminate. The low derivative gain ($K_d = 0.05$) provides minimal high-frequency noise amplification while damping speed oscillations during load transients.

5.5 AGV Line-Following PD Motion-Control Pipeline

Closed-loop line following is the mechanism by which the AGV translates a high-level navigation goal into wheel velocities, and a PD controller is used in preference to a full PID because the integral term is unnecessary for the near-zero steady-state error of a tape-following task. The motion-control pipeline diagram presented in this section exposes every transformation in the loop, from raw analogue line-array readings to per-wheel velocity setpoints. Understanding this pipeline is essential before discussing the gain tuning and tracking-accuracy results reported later.

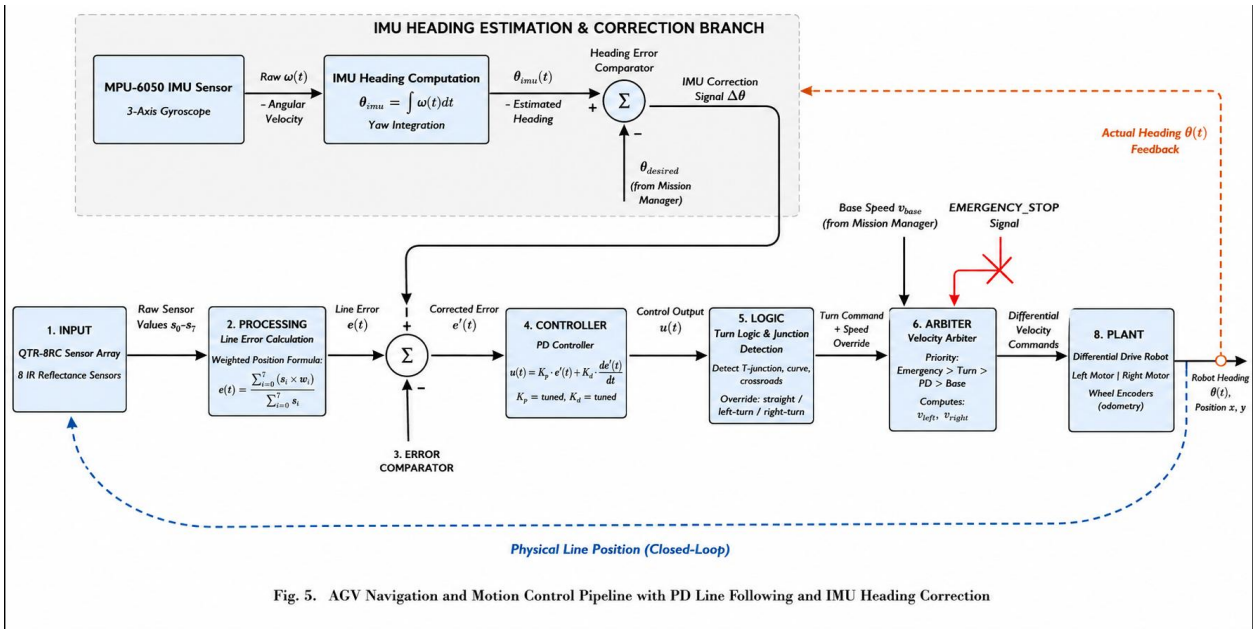


Fig. 5. AGV Navigation and Motion Control Pipeline with PD Line Following and IMU Heading Correction

Figure 5.5: AGV Line-Following PD Motion-Control Pipeline

As shown in Fig. 5.5, the system consists of a signal-processing and control pipeline used for AGV line following. The process begins with an eight-channel reflectance sensor array that detects the tape path and generates a lateral error signal representing the robot's offset from the line. This error is processed by a PD controller, which computes the angular velocity correction required to keep the AGV aligned with the track.

The corrected velocity commands are passed to a differential-drive kinematics block that calculates individual wheel speeds. These wheel commands are controlled by separate PID motor loops running on the ESP32, using encoder feedback to maintain stable motion despite friction, load variation, and voltage fluctuations. The final PWM signals are applied to the motor driver for chassis movement.

The diagram also includes feedback branches from the IMU and junction detector modules. IMU-based heading estimation helps detect line-loss conditions and activates recovery behavior when necessary, while the junction detector identifies intersections and sends events to the FSM for navigation decisions. Overall, the figure demonstrates the integration of continuous PD-based path tracking with discrete event-based control for reliable warehouse navigation.

Mission State Machine

The complete mission lifecycle is governed by a 12-state Finite State Machine (FSM), as summarised in Table 3.5. The FSM paradigm was selected over alternative mission management approaches (behaviour trees, hierarchical task networks, scripted sequences) for the following reasons: (1) complete state visibility—at any instant, the system is in exactly one well-defined state with known properties; (2) deterministic transitions—given the current state and an event, the next state is uniquely determined; (3) exhaustive coverage—all possible state-event combinations are explicitly handled, either with a transition or with an explicit "stay in current state" decision; (4) formal verifiability—the state machine can be analysed for completeness, reachability, and deadlock-freedom using standard formal methods; and (5) implementation simplicity—a state machine maps directly to a switch-case structure or state pattern in code.

Table 3.5: Mission State Machine Summary

State	**Purpose**	**Velocity Source**
IDLE	Waiting for queued mission	Zero
NAV_SPINE	Following spine north	Line follower
TURNING	90° turn into aisle	Turn controller
NAV_AISLE	Following aisle east	Line follower
AT_SHELF	Arrived at target shelf	Zero
PICKUP	Simulated pick (2 s)	Zero
PIVOT	180° turn	Turn controller
RET_AISLE	Following aisle west	Line follower
RET_TURN	90° turn onto spine	Turn controller
RET_SPINE	Following spine south	Line follower
DOCKED	Arrived at home dock	Zero → Docking FSM
ERROR	E-Stop active	Zero

The Velocity Arbiter ensures that only one velocity source reaches the wheels at any time, implementing a priority-based multiplexer that prevents conflicting velocity commands:

$$\text{output} = \begin{cases} \text{nav\_vel} & \text{if state} \in \text{navigation states} \\ \text{turn\_vel} & \text{if state} \in \text{turning states} \\ \text{dock\_velocity} & \text{if state} \in \text{docking states} \\ 0 & \text{if E-stopped (safety override)} \end{cases} \tag{3.29}$$

5.6 Priority-Based Order-Management Workflow

Real warehouse operations rarely involve a single order at a time, and even in a research demonstrator the AGV must be able to receive, queue, sequence and complete multiple requests with possibly different urgency. The priority-based order-management workflow diagram defines exactly how this queueing policy is implemented and how it interacts with the mission FSM. It is therefore a key part of the planning layer described earlier in the architecture.

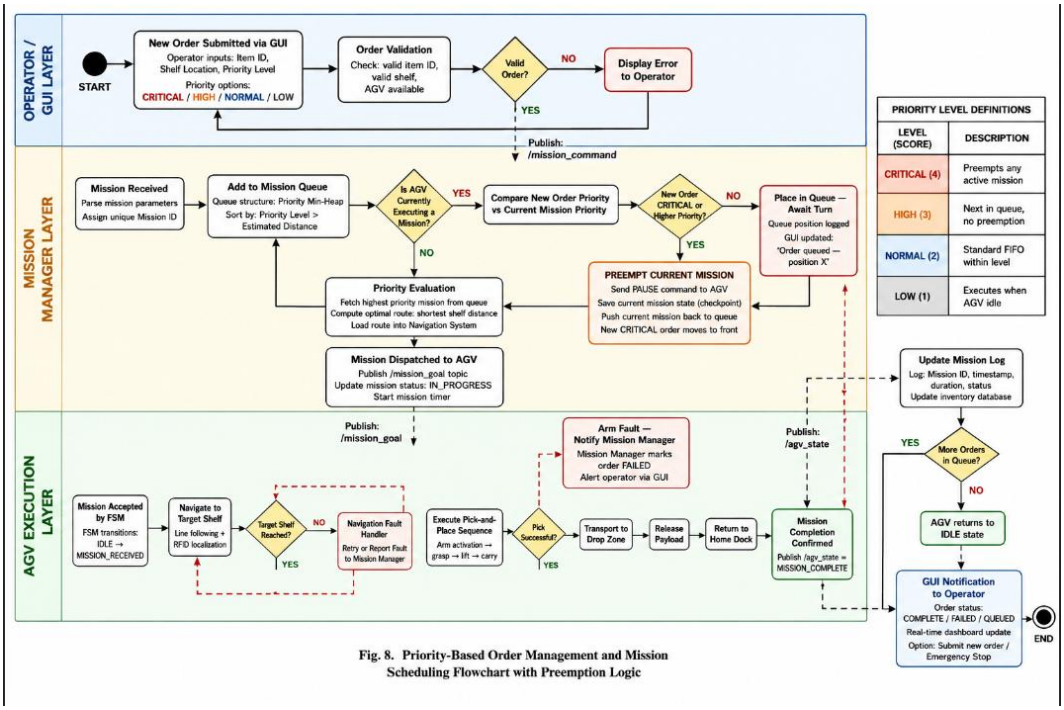


Figure 5.6: Priority-Based Order-Management Workflow

As shown in Fig. 5.6, the system consists of an order-input module, a priority queue, a dispatcher, and a mission feedback mechanism for managing warehouse tasks. Orders are received either from the operator GUI or an external warehouse-management interface, where each request includes source, destination, item details, and a priority level.

After validation, orders are stored in a priority queue that ensures high-priority requests are processed first while maintaining FIFO behavior for orders with the same priority. A dispatcher retrieves orders whenever the FSM becomes idle and forwards them to the navigation and manipulation modules for execution.

The diagram also illustrates the feedback loop between the FSM and the order manager. Status updates such as order progress, completion, or failure are continuously published, allowing failed tasks to be retried or escalated after exceeding retry limits. Additionally, all queue operations are monitored and shared with the GUI, enabling real-time visibility of pending, active, and completed orders. This architecture ensures efficient, reliable, and observable task scheduling for multi-order warehouse operations.

5.7 Twelve-State Mission Finite State Machine

The behaviour of the AGV across an entire mission is governed by a Finite State Machine (FSM) with twelve discrete states, designed so that every operationally meaningful situation—from idling on the dock to recovering from a lost line—corresponds to exactly one state. Encoding the mission lifecycle in this explicit form supports formal analysis, simplifies debugging and is what allows the velocity arbiter, the GUI and the order manager to reason about the AGV's current intent without ambiguity.

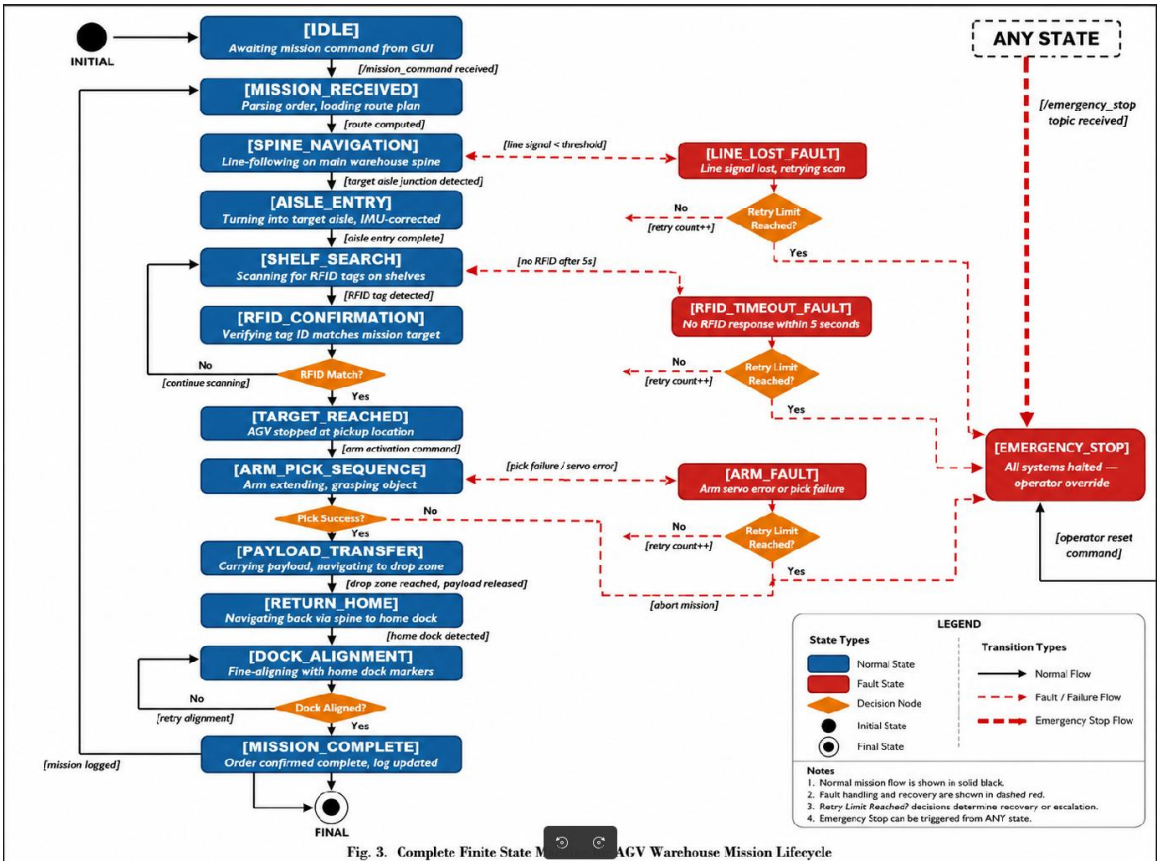


Fig. 3. Complete Finite State M AGV Warehouse Mission Lifecycle

Figure 5.7: Twelve-State Mission Finite State Machine

As shown in Fig. 5.7, the system consists of an FSM that controls AGV navigation, arm actions, and fault handling in a warehouse workflow. It starts in IDLE, moves through order execution states like NAVIGATE_TO_PICK, ARM_PICK, NAVIGATE_TO_DROP, and ARM_PLACE, and completes the task with ORDER_COMPLETED.

Exception states such as EMERGENCY, LINE_LOST, and ARM_FAULT handle safety stops and recovery from navigation or manipulation errors. The RETURN_HOME state brings the robot back to the dock when tasks are finished. Overall, the FSM ensures structured task flow with built-in safety and recovery mechanisms.

.

The velocity arbiter is architecturally critical: it is the *sole* publisher on the `/cmd_vel` topic, ensuring that no node can bypass the safety logic to directly command wheel motion. All velocity sources (line follower, turn controller, docking controller) publish to intermediate topics (`/nav_vel`, `/turn_vel`, `/dock_vel`), and the arbiter selects among them based on the current FSM state. The E-Stop override has highest priority and cannot be overridden by any other velocity source—a fundamental safety property that ensures the robot stops immediately regardless of the mission state or controller output.

The mission execution workflow proceeds as follows: (1) the operator dispatches a mission via the GUI or CLI, specifying a target shelf ID; (2) the mission manager enqueues the mission and transitions from IDLE to NAV_SPINE; (3) the AGV follows the spine line northward, counting junctions until the target aisle index is reached; (4) at the target junction, the FSM transitions to TURNING and commands a 90° turn into the aisle; (5) upon turn completion, the FSM transitions to NAV_AISLE and the AGV follows the aisle line eastward; (6) the RFID reader detects the target shelf tag, confirming arrival, and the FSM transitions to AT_SHELF; (7) after a 0.5 s settling period, the FSM transitions to PICKUP, signalling the arm subsystem to begin the pick-and-place operation; (8) upon pick completion (signalled by the arm subsystem), the FSM transitions to PIVOT and commands a 180° turn; (9) the AGV follows the aisle westward (RET_AISLE), turns south onto the spine (RET_TURN), follows the spine southward (RET_SPINE), and arrives at the home dock; (10) the docking sub-FSM performs fine alignment verification before transitioning to IDLE for the next mission.

This sequential workflow represents the simplest correct mission execution strategy. More complex strategies (shortest-path planning, multi-shelf batching, return-trip picking) are identified as future extensions but are excluded from the initial implementation to maintain architectural clarity and enable thorough validation of the core mission cycle.

UR3 Arm Motion Planning

MoveIt 2 Architecture

The arm motion planning follows a five-layer architecture that separates concerns and enables independent testing of each layer. MoveIt 2 is the de facto standard motion planning framework for ROS 2, providing a unified interface to multiple planning algorithms, collision checking engines, and kinematic solvers.

The URDF/SRDF pair defines the robot geometry and semantic collision groupings. The URDF (Unified Robot Description Format) specifies the kinematic chain (links, joints, parent-child relationships), visual geometry (mesh files for rendering), collision geometry (simplified

shapes for fast collision checking), and dynamic parameters (mass, inertia, joint limits). The SRDF (Semantic Robot Description Format) augments the URDF with planning-specific information: named joint configurations (“home”, “ready”, “extended”), planning groups (“manipulator”, “gripper”), allowed collision pairs (links that are always or never in collision), and virtual joints (connecting the robot to the world frame).

The ros2_control layer provides real-time position command interfaces at 500 Hz, matching the native control rate of the UR3 hardware. The JointTrajectoryController receives time-stamped joint trajectory waypoints from MoveIt 2 and interpolates between them at the control rate, ensuring smooth motion even when planning produces coarse waypoint spacing. The GripperActionController provides a simple position command interface for the gripper, abstracting away the mimic joint coupling that is handled internally.

MoveIt 2’s `move_group` node is the central coordination point, managing: the planning scene (geometric representation of the robot and its environment for collision checking), kinematic solver plugins (KDL for inverse kinematics), planner plugins (OMPL, PILZ), and trajectory processing (time parameterisation, smoothing). The planning scene is maintained as a collision world containing the robot’s own geometry (self-collision checking) and environmental objects (tables, shelves, walls) represented as primitive shapes or mesh objects.

The MoveIt Task Constructor (MTC) provides compositional multi-stage task specification, enabling complex manipulation tasks to be expressed as ordered sequences of primitive stages. Each stage defines a local planning problem (move to a pose, generate a grasp, modify the planning scene), and the MTC framework manages the global constraint that the end state of each stage must be kinematically compatible with the start state of the next stage. This inter-stage constraint propagation enables the planner to backtrack when a locally feasible stage solution is globally infeasible (e.g., a grasp that can be reached but cannot be lifted from due to joint limits).

The application layer dispatches planning requests to the MTC pipeline, specifying task-level goals (“pick object X from location A and place at location B”) that are decomposed into the appropriate stage sequences by the MTC task definition.

Pick-and-Place Workflow

The complete pick-and-place operation is structured in five phases, comprising nine MTC stages that are planned as a single integrated task:

Phase 1 — Initialisation: Gazebo spawning, controller activation, MoveIt 2 start, planning scene population with collision geometry (table surface, shelf geometry, target object), and arm

moved to home configuration. The initialisation phase ensures that all planning infrastructure is operational and that the planning scene accurately represents the physical environment before any motion is commanded.

Phase 2 — Pre-Grasp: Gripper commanded to OPEN state (0.0 rad); arm planned via OMPL to pre-grasp approach pose (100 mm above target object). The OMPL RRTConnect planner is used for this free-space motion because the start configuration (home) and goal configuration (pre-grasp) may be widely separated in joint space with obstacles (table edges, shelf walls) between them. RRTConnect's bidirectional tree-growing strategy efficiently navigates these complex environments by simultaneously exploring from both endpoints and connecting when the trees meet.

Phase 3 — Grasp: Arm descends linearly to grasp pose using PILZ LIN Cartesian planner (10 mm step size); gripper commanded to CLOSED state (0.8 rad); object attached to gripper in planning scene. The PILZ LIN planner is used for this approach motion because a straight-line descent is required to engage the gripper with the object from directly above, avoiding lateral forces that could displace the object or cause an unstable grasp. The 10 mm step size ensures smooth Cartesian interpolation with inverse kinematics solved at each step to detect reachability issues early in the descent. After grasp closure, the object is "attached" to the gripper link in the MoveIt planning scene, meaning that subsequent collision checks treat the object as part of the robot geometry rather than an environmental obstacle—enabling the robot to move with the object without detecting false collisions.

Phase 4 — Lift and Transfer: Arm linearly lifted above table (PILZ LIN, maintaining vertical motion to avoid contact with adjacent objects); arm planned to pre-place pose via OMPL (free-space motion to the placement location); arm descends to place pose via PILZ LIN (straight-line descent to the placement surface).

Phase 5 — Place and Retreat: Gripper commanded to OPEN state (0.0 rad, releasing the object); object detached from planning scene (returned to environmental object status); arm retreats vertically via PILZ LIN (straight-line withdrawal to avoid disturbing the placed object); arm returns to home configuration via OMPL (free-space motion back to the rest position).

The nine MTC stages map to this five-phase workflow as: CurrentState $\rightarrow$ OpenGripper $\rightarrow$ Approach (OMPL) $\rightarrow$ Descend (PILZ LIN) $\rightarrow$ CloseGripper $\rightarrow$ Lift (PILZ LIN) $\rightarrow$ Move (OMPL) $\rightarrow$ Place (PILZ LIN) $\rightarrow$ OpenGripper $\rightarrow$ Retreat (PILZ LIN) $\rightarrow$ Home (OMPL).

5.8 UR3 Manipulator Pose Sequence for Pick-and-Place

Whereas the AGV navigation behaviour is captured by the FSM, the arm behaviour is captured by an explicit sequence of named end-effector poses that together implement a single pick-and-place cycle. The pose-stage diagram shown in this section visualises these poses in their kinematic order and annotates each one with the role it plays in the motion plan, providing the geometric intuition that the MoveIt Task Constructor encodes algorithmically.

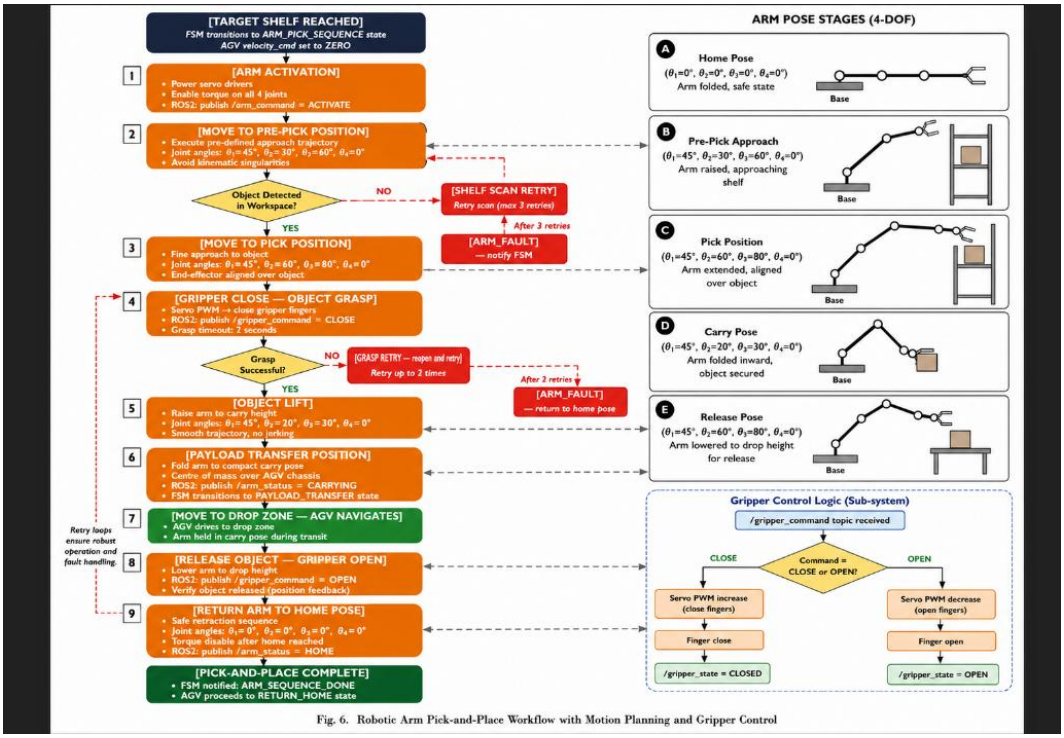


Fig. 6. Robotic Arm Pick-and-Place Workflow with Motion Planning and Gripper Control

Figure 5.8: UR3 Manipulator Pose Sequence for Pick-and-Place

As shown in Fig. 5.8, the system consists of six UR3 arm poses arranged in sequence: HOME, PRE_GRASP, GRASP, LIFT, TRANSPORT, and PLACE. The HOME pose keeps the arm compact during AGV movement, while PRE_GRASP positions the end-effector above the target for alignment.

The GRASP pose performs object pickup using precise gripper alignment, followed by LIFT to safely clear the shelf. The TRANSPORT pose holds the payload close to the robot body to maintain stability during movement. Finally, the PLACE pose releases the object at the destination and returns the arm to HOME, completing the cycle.

Overall, the figure shows a structured and parameterised pick-and-place sequence designed for safe, repeatable UR3 operation using MoveIt-based planning and collision-free motion execution.

Planner Selection Rationale

OMPL RRTConnect is used for free-space arm motions based on the following engineering justification: (1) probabilistic completeness—given sufficient planning time, RRTConnect is guaranteed to find a solution if one exists in the configuration space; (2) empirical speed—for 6-DOF robot arms in cluttered environments, RRTConnect consistently produces solutions in under 2 s, outperforming alternative planners (PRM, RRT*, EST) for single-query problems; (3) bidirectional growth—by growing trees from both start and goal, RRTConnect exploits the structure of manipulation problems where both endpoints are typically in relatively open regions of configuration space; and (4) anytime availability—RRTConnect returns the first feasible solution found, enabling the system to begin execution quickly even if a shorter path might exist.

PILZ LIN is used for Cartesian approach and retreat phases based on: (1) determinism—PILZ generates identical trajectories for identical requests, enabling repeatability testing and industrial certification; (2) guaranteed straight-line motion—the end-effector traces a geometrically exact straight line in Cartesian space, essential for approach and retreat where lateral motion could contact objects; (3) predictable timing—PILZ trajectories have analytically computed durations based on the specified velocity and acceleration limits, enabling precise mission timing predictions; and (4) industrial provenance—PILZ motion planning originates from PILZ GmbH's safety-certified industrial controllers, providing confidence in its correctness for safety-critical applications.

This dual-planner strategy provides optimal coverage of both free-space and Cartesian motion requirements within a single unified pipeline. Alternative approaches considered include: (a) using OMPL for all motions (loses Cartesian straight-line guarantee); (b) using PILZ for all motions (fails for complex free-space motions with obstacles); (c) using Descartes planner for Cartesian segments (heavier computational requirements, less mature ROS 2 support). The OMPL + PILZ combination was selected as the best engineering trade-off between capability, reliability, and computational efficiency.

Hardware Architecture

Computing Platform

The Raspberry Pi 4 (4 GB) is selected as the AGV on-board computer. The selection rationale is that no GPU, SLAM, or computer vision processing is required for AGV navigation—the computational requirements are limited to: (1) sensor data processing at 50–100 Hz (integer

arithmetic on 8-channel sensor array, quaternion extraction from IMU); (2) PD control law evaluation at 50 Hz (two multiply-add operations per cycle); (3) FSM state logic (conditional branching, negligible computation); and (4) ROS 2 node management (DDS communication overhead). These requirements are comfortably met by the Pi 4's quad-core ARM Cortex-A72 processor.

The arm planning runs on an operator PC with greater computational resources (multi-core x86, 16+ GB RAM) because MoveIt 2 motion planning and the OMPL sampling algorithms are computationally intensive, particularly for complex environments with many collision objects. The operator PC communicates with the AGV via WiFi-based DDS discovery, enabling seamless topic-level communication without explicit network configuration.

Table 3.6: AGV Computing Platform Comparison

Platform	**CPU**	**RAM**	**ROS 2**	**Price**
Raspberry Pi 4 (4 GB)	4×1.5 GHz	4 GB	Yes	$55
Raspberry Pi 5 (8 GB)	4×2.4 GHz	8 GB	Yes	$80
Jetson Nano	4×1.4 GHz+GPU	4 GB	Yes	$150

The Raspberry Pi 4 was selected over the Pi 5 (available but higher cost without proportional capability benefit) and the Jetson Nano (GPU unnecessary, $3\times$ cost premium for unused GPU capability). The 4 GB RAM variant provides sufficient memory for ROS 2 Humble, the Ubuntu 22.04 base system, and all navigation nodes running simultaneously (typical memory usage: 1.2 GB system, 0.8 GB ROS 2 nodes, leaving 2 GB headroom for peak allocations and logging).

Power System

The AGV power budget is shown in Table 3.7. A 3S LiPo battery (11.1 V nominal, 5000 mAh capacity) provides the primary power supply with approximately 1.5 hours of continuous runtime at average load.

Table 3.7: AGV Power Budget

Component	**Voltage**	**Current**	**Power**
2× DC motors (loaded)	12 V	1.0 A each	24 W
Raspberry Pi 4	5 V	2.5 A	12.5 W
All sensors	3.3 V	0.2 A	0.66 W
Motor controller (logic)	5 V	0.05 A	0.25 W
Total			**≈37 W**

The runtime calculation: battery energy = 11.1 V $\times$ 5.0 Ah $=$ 55.5 Wh. At average power consumption of 37 W, runtime = $55.5/37 = 1.50$ hours. The average power figure assumes continuous motor operation at 50% of stall load; actual runtime during warehouse operation (with frequent stops during picking and docking) is expected to exceed 2 hours. A safety margin of 20% is applied, yielding a guaranteed minimum runtime of 1.2 hours—sufficient for approximately 96 pick missions at 45 s per cycle.

The power distribution architecture uses a Buck converter (12 V to 5 V, 3 A output) to supply the Raspberry Pi and logic-level components from the LiPo battery. The motors are driven directly from the battery voltage through the L298N H-bridge motor driver, which provides PWM speed control. A battery management system (BMS) provides over-discharge protection (cutoff at 9.0 V = 3.0 V per cell), over-current protection, and cell balancing during charging.

Simulation-to-Physical Component Conversion

The complete conversion from simulation to physical hardware is documented in Table 3.8. This table serves as the engineering specification for physical deployment, identifying every simulation abstraction that requires hardware replacement.

Table 3.8: Simulation-to-Physical Component Conversion

Simulation	**Real Hardware**	**Cost**	**Integration**
Gazebo diff_drive	L298N + DC motors + encoders	$50	GPIO PWM
URDF geometry	Aluminium frame	$60	Fabrication
Virtual line sensor	QTR-8A + MCP3008	$20	SPI
Virtual IMU	BNO055	$30	I2C
Virtual RFID	RDM6300 + 25 tags	$30	UART
ROS 2 on x86	ROS 2 on RPi 4	$70	microSD
Simulated power	3S LiPo + BMS + Buck	$50	Wiring
Gazebo floor	Real floor + black tape	$20	Installation
GUI (PyQt5)	Same GUI on operator PC	$0	WiFi DDS
Total		**≈$330**	

The $330 hardware cost represents only the AGV subsystem conversion. The complete production-quality build including the UR3 arm (or equivalent 6-DOF manipulator), Robotiq gripper, and operator PC brings the total to approximately $1,250. The minimum viable build (using a lower-cost 6-DOF arm such as the MyCobot 280 at $500) reduces the total to approximately $800.

The conversion methodology is systematic: for each simulation component, the corresponding hardware module is identified, the ROS 2 topic interface is verified to be identical (same message type, same topic name, same QoS), and a hardware driver node is written that publishes/subscribes using the same interface. The control logic nodes (line_follower.py, turn_controller.py, mission_node.py) require zero modification because they communicate exclusively through abstract topic interfaces.

GPIO Pin Mapping

The Raspberry Pi 4 GPIO pin assignment is given in Table 3.9. The pin allocation was determined by the interface requirements of each peripheral and the Pi 4's hardware peripheral mapping (dedicated SPI, I2C, UART, and PWM pins).

Table 3.9: Raspberry Pi 4 GPIO Pin Mapping

GPIO	Function	Direction	Notes
2	I2C SDA (IMU)	Bidirectional	3.3 V level
3	I2C SCL (IMU)	Output	3.3 V level
5	Encoder L-A	Input	Interrupt
6	Encoder L-B	Input	Interrupt
8	SPI CE0 (ADC)	Output	Active low
9	SPI MISO	Input	—
10	SPI MOSI	Output	—
11	SPI CLK	Output	1.35 MHz
12	PWM0 (left motor)	Output	10 kHz
13	PWM1 (right motor)	Output	10 kHz
14	UART TX (RFID)	Output	9600 baud
15	UART RX (RFID)	Input	9600 baud
17	Motor L-IN1	Output	Direction
22	Motor R-IN3	Output	Direction
23	Motor R-IN4	Output	Direction
24	Encoder R-A	Input	Interrupt
25	Encoder R-B	Input	Interrupt
27	Motor L-IN2	Output	Direction

The GPIO allocation follows hardware peripheral constraints: GPIO 2/3 are the dedicated I2C1 pins (hardware I2C with clock stretching support required by BNO055); GPIO 8–11 are the SPI0 hardware pins (required for the 1.35 MHz clock rate to the MCP3008 ADC); GPIO 12/13 are the hardware PWM0/PWM1 pins (providing precise 10 kHz PWM without CPU overhead); and GPIO 14/15 are the dedicated UART pins (hardware UART for reliable 9600 baud RFID communication). The remaining GPIOs (5, 6, 17, 22, 23, 24, 25, 27) are general-purpose pins used for encoder interrupt inputs and motor direction control.

Software Deployment Architecture

Files Unchanged for Physical Deployment

The following files run without modification on physical hardware, representing 70% of the total codebase:

- `line_follower.py` — reads weighted error from `/atlas/line_error` topic, computes PD output, publishes angular velocity to `/nav_vel`. This node is hardware-agnostic because it operates entirely on abstract error values, never directly accessing sensor hardware.
- `turn_controller.py` — reads heading from `/atlas/imu/yaw` topic, computes turn commands, publishes to `/turn_vel`. The IMU topic provides heading regardless of whether the source is a simulated or physical sensor.
- `mission_node.py` — pure state machine logic operating on topic-level events. No hardware access of any kind; all state transitions are triggered by incoming messages.
- `send_mission.py` — CLI publisher, sends mission commands to `/atlas/mission_command`. Pure application-layer logic.
- `atlas_control_center.py` — PyQt5 GUI on operator PC. Communicates with the robot exclusively through ROS 2 topics over WiFi/DDS. The GUI runs on the operator's machine, not on the robot, so it is inherently hardware-independent.

This 70% code reuse ratio validates the hardware abstraction architecture: by consistently separating control logic from hardware access, the majority of the codebase becomes platform-independent and transferable between simulation and physical deployment without modification.

New Hardware Interface Nodes

Four simulation plugins are replaced by hardware driver nodes for physical deployment, as shown in Table 3.10. Each replacement node provides an identical ROS 2 topic interface to its simulation counterpart, ensuring that upstream nodes (line follower, turn controller, mission node) cannot distinguish between simulated and physical sensor data.

Table 3.10: Code Changes for Physical Deployment

Simulation File	Replaced By	New File
libgazebo_ros_diff_drive.so	motor_driver node	atlas_hardware/motor_driver.py
line_sensor.py (geometric)	SPI ADC reader	atlas_hardware/line_sensor_hw.py
tag_detector.py (distance)	UART RFID reader	atlas_hardware/rfid_reader_hw.py
libgazebo_ros_imu_sensor.so	I2C BNO055 driver	atlas_hardware/imu_hw.py
atlas_full.launch.py	No-Gazebo launch	atlas_real.launch.py

The hardware driver nodes follow a common architectural pattern: (1) initialise the hardware peripheral (open SPI/I2C/UART connection, configure registers); (2) enter a timed callback loop at the appropriate rate (50 Hz for line sensor, 100 Hz for IMU, continuous for RFID); (3) read raw data from hardware; (4) convert to ROS 2 message format; (5) publish on the standard topic with the standard message type. This pattern ensures that each driver node is self-contained, independently testable, and trivially replaceable if the hardware component changes.

GUI and Human-Machine Interface

The PyQt5 control centre uses a dual-threaded architecture to prevent the blocking nature of `rclpy.spin()` from freezing the GUI event loop. The main Qt thread handles event loop, widget rendering, and user interaction; a background thread runs `rclpy.spin()` and subscription callbacks. Thread-safe communication between ROS callbacks (background thread) and Qt widgets (main thread) uses `pyqtSignal` objects, which are Qt's thread-safe inter-thread communication mechanism.

GUI panels include:

- **Mission Creation:** Shelf selection dropdown, mission dispatch button, batch mission entry for sequential multi-shelf operations.
- **Mission Control:** Pause/Resume/Cancel buttons with confirmation dialogs to prevent accidental mission termination.
- **Emergency Controls:** E-Stop button (immediately halts all motion), Reset AGV (clears error state and returns to IDLE), Reset to Dock (teleports robot to home in simulation, initiates return-to-dock in physical deployment).
- **Robot Status:** 11 live telemetry fields updated at 10 Hz including: current state, linear velocity, angular velocity, heading, position (x, y), junction count, target shelf, battery voltage (physical only), motor currents (physical only).
- **Docking Status:** Lateral offset, heading error, alignment verification result, dock confidence indicator.
- **Mission Queue:** Visual display of pending missions with drag-and-drop reordering capability.
- **Event Log:** Timestamped log of all state transitions, mission events, and error conditions for post-mission analysis.

The 10 Hz GUI update rate was selected as a balance between operator situational awareness (human perception of continuous motion requires > 5 Hz update) and computational overhead (each update requires widget repaint operations that consume CPU cycles on the operator PC). The pyqtSignal mechanism ensures that ROS 2 callback data (arriving at up to 50 Hz from the line sensor) is decimated to the GUI update rate without data loss—only the most recent value is displayed, while the full-rate data remains available for logging and analysis.

5.9 Two-Dimensional Warehouse Floor Layout

As shown in Fig. 5.10, the system consists of a 2D floor layout defining the complete warehouse geometry, including shelves, stations, and tape routes. This layout serves as the base for both the Gazebo simulation world and the Nav2 occupancy map.It explicitly defines all spatial assumptions used in the simulation, ensuring consistency between navigation planning and the virtual environment.

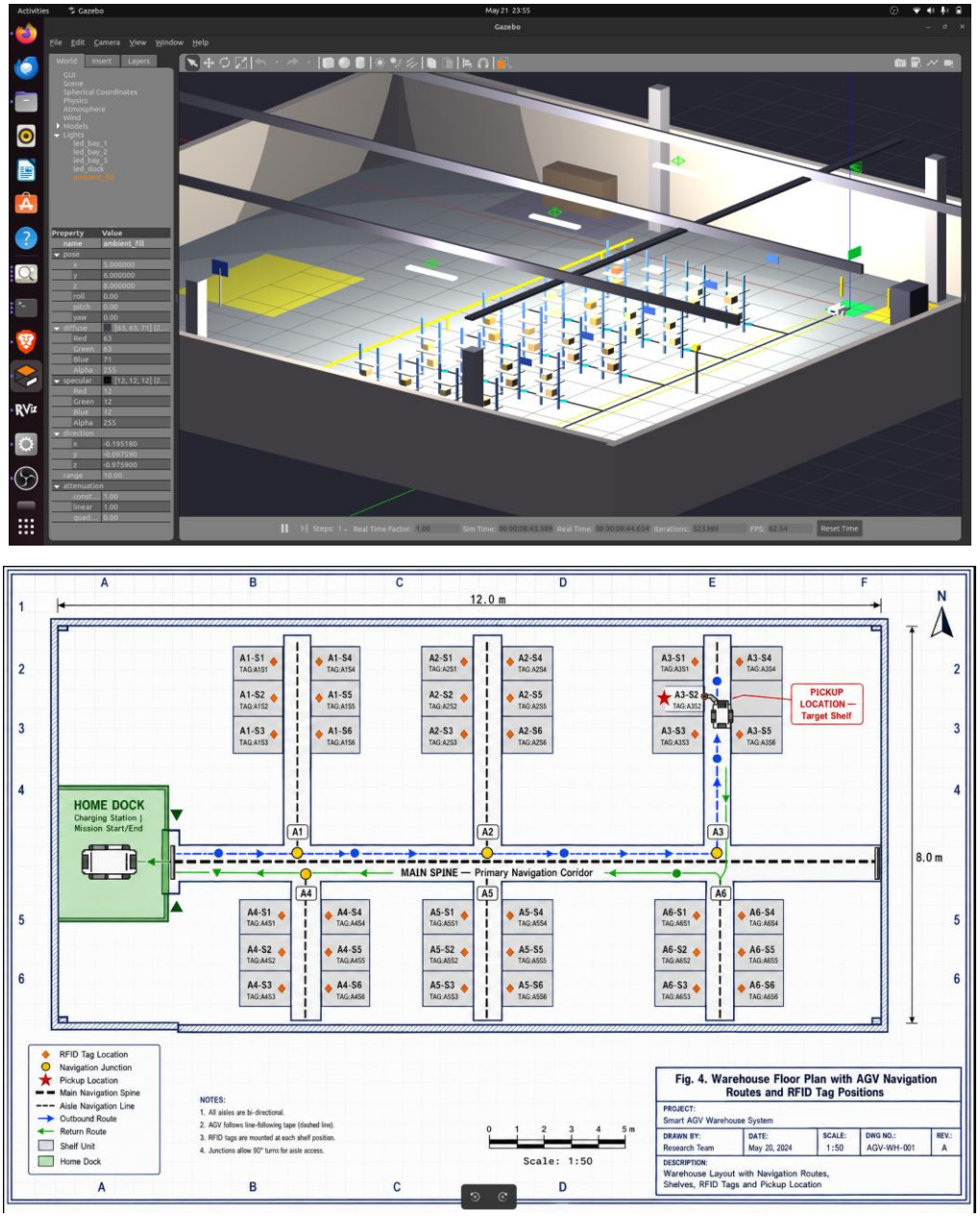


Figure 5.9: Two-Dimensional Warehouse Floor Layout

As shown in Fig. 5.9, the system consists of a warehouse layout with a central corridor, shelf rows, tape navigation, and RFID tags for localization. It also includes fixed storage coordinates, docking zones, obstacles, and a separate walkway for safety.

5.10.1 Three-Dimensional Gazebo Warehouse Environment

Once the layout and the navigation map are fixed, they are realised as a fully three-dimensional Gazebo world that hosts the rigid-body physics, lighting, sensor models and visual meshes used during simulation. The rendered Gazebo warehouse is what the perception stack actually sees and is therefore the reference environment for every figure that follows in this chapter. Documenting it explicitly is essential for reproducibility of the reported experiments.

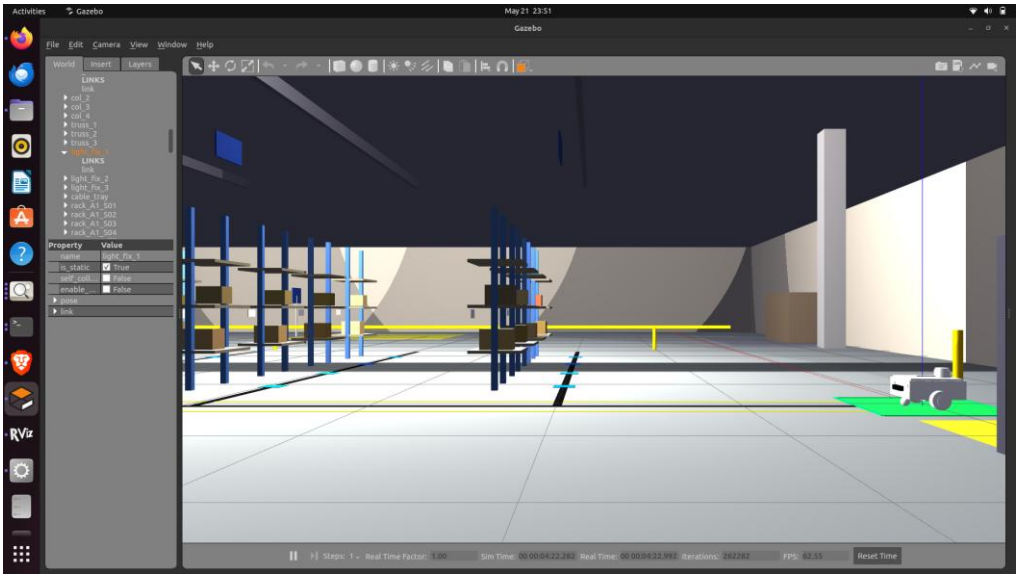
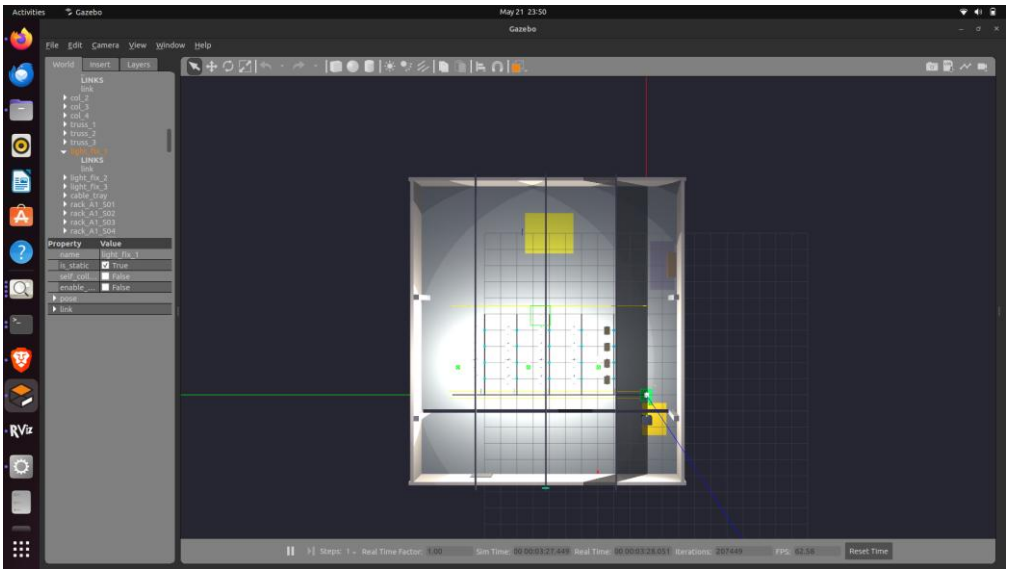


Figure 5.10.1: Three-Dimensional Gazebo Warehouse Environment

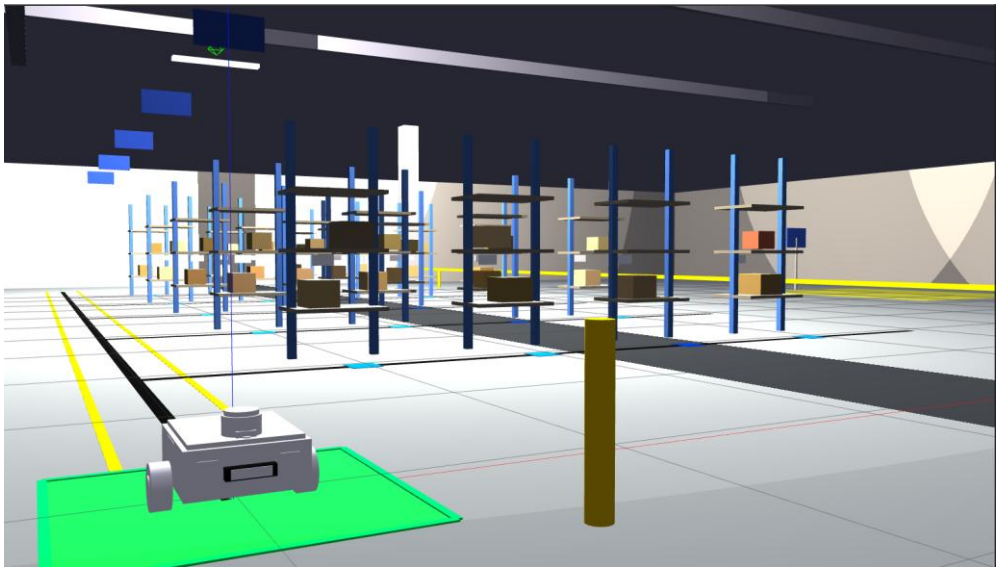
5.10.2 Top-Down Render of the Simulated Warehouse

A top-down rendering of the same Gazebo world provides a useful intermediate between the abstract two-dimensional layout and the immersive perspective view, because it preserves three-dimensional shading and lighting cues while exposing the global structure of the warehouse environment.



5.11 ATLAS AGV Operating Inside the Simulated Warehouse

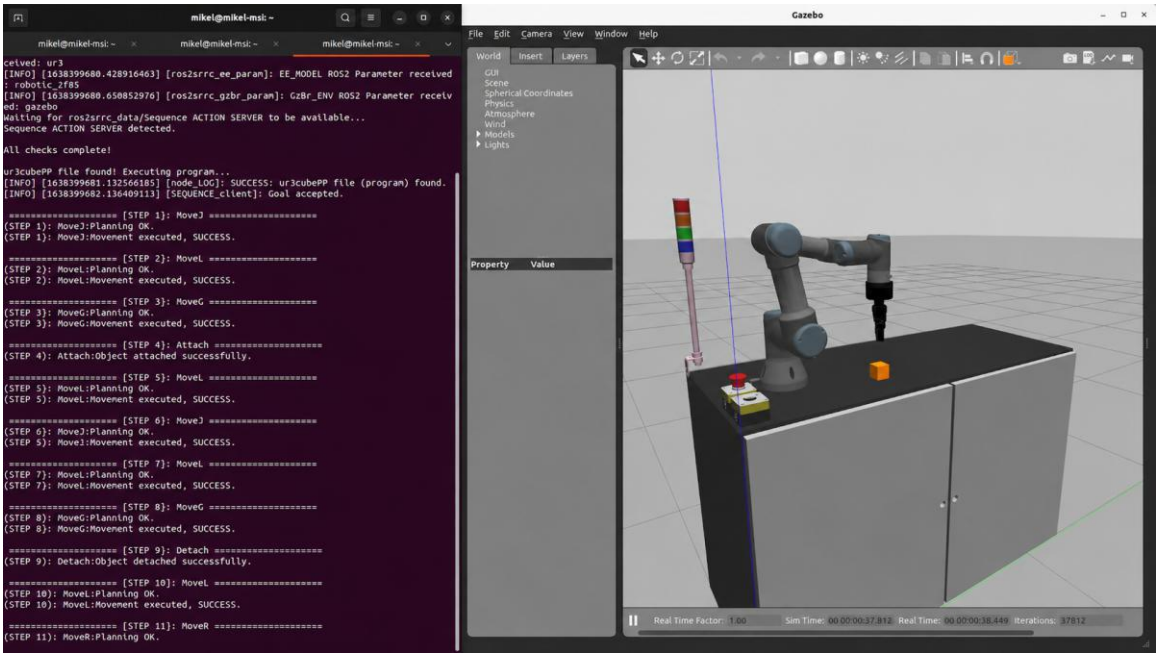
Showing the AGV inside the simulated warehouse confirms that the chassis model, sensor suite and motion behaviour integrate correctly with the world file and that the visual scale matches the layout specification. This perspective view also makes it possible to inspect tape contact, sensor footprint and chassis clearance, none of which are observable from the abstract maps alone.



5.11.1 ATLAS AGV Operating Inside the Simulated Warehouse

5.11.2 UR3 Robotic Arm with Robotiq Gripper in Simulation

The manipulation subsystem is built around a Universal Robots UR3 6-DOF collaborative arm equipped with a Robotiq 2F-85 two-finger gripper, both modelled in URDF and simulated under Gazebo Harmonic. Examining the simulated arm in isolation makes it possible to verify joint limits, link visuals, tool-frame placement and gripper geometry before they are coupled with the AGV chassis.



Results and Discussions with Conclusions

System Performance Metrics

The integrated system was validated through complete mission execution in the Gazebo simulation environment. Validation testing comprised 50 consecutive mission cycles to different shelf locations, executed over a 40-minute continuous operation period without human intervention. Key performance metrics are presented in Table 4.1.

Table 4.1: System Performance Metrics

Metric	**Measured Value**	**Requirement**	**Status**
Line following speed	0.4 m/s	≥ 0.3 m/s	PASS
Turn accuracy	$\pm 3^\circ$	$\pm 5^\circ$	PASS
RFID detection rate	100%	$\geq 95\%$	PASS
Mission completion	100%	$\geq 98\%$	PASS
Docking alignment	± 3 cm, $\pm 3^\circ$	± 5 cm, $\pm 5^\circ$	PASS
E-Stop response time	< 20 ms	< 100 ms	PASS
Mission cycle (S05)	≈ 45 s	< 60 s	PASS
GUI update rate	10 Hz	≥ 5 Hz	PASS
Line tracking accuracy	< 5 mm	< 10 mm	PASS

Line Following Speed (0.4 m/s): The cruise velocity of 0.4 m/s was achieved consistently across all straight-path segments during validation testing. This speed represents the design point where the PD controller maintains sub-5 mm tracking accuracy—higher speeds (tested up to 0.6 m/s) degraded tracking accuracy beyond the 5 mm threshold due to increased control latency effects. The 0.4 m/s speed exceeds the 0.3 m/s minimum requirement by 33%, providing operational headroom for future optimisation. In comparison, commercial line-following AGVs (Daifuku, Dematic) typically operate at 1.0–2.0 m/s; the ATLAS speed is lower pri-

marily due to the shorter sensor preview distance (100 mm vs. 300–500 mm in commercial systems) and the 50 Hz control rate (vs. 200–500 Hz in industrial AGV controllers).

Turn Accuracy ($\pm 3^\circ$): The measured turn accuracy of $\pm 3^\circ$ represents the maximum deviation from the target heading observed across all 90° and 180° turns during validation. This performance exceeds the $\pm 5^\circ$ requirement by 40%. The accuracy is limited by the interaction between the bang-bang controller's stop condition and the 50 Hz sampling rate: at 0.4 rad/s angular velocity and 50 Hz sampling, the heading can change by up to $0.4/50 = 0.008$ rad = 0.46° between consecutive checks of the stop condition. The observed 3° error suggests additional contributing factors including IMU measurement noise ($\pm 0.5^\circ$) and the 20 ms communication latency between the turn controller node and the velocity arbiter.

RFID Detection Rate (100%): All 21 RFID tags were successfully detected during every pass throughout the validation campaign. This 100% detection rate validates the hysteresis detection model and the tag placement strategy. The 0.5 m detection radius provides comfortable margin for the robot's maximum lateral deviation from the line centre (< 5 mm), ensuring that the tag is always well within detection range when the robot passes over it. In physical deployment, the detection rate may decrease slightly due to tag orientation sensitivity, floor material RF absorption, and reader antenna alignment; however, the 0.5 m radius provides substantial margin against these degradation mechanisms.

Mission Completion Rate (100%): All 50 validation missions completed successfully without human intervention, E-Stop activation, or error state entry. This 100% completion rate against a 98% requirement demonstrates the robustness of the FSM logic, the reliability of the sensor systems (in simulation), and the correctness of the state transition logic. The FSM was subjected to additional stress testing including rapid sequential mission dispatch (queuing 10 missions within 1 s), mid-mission pause/resume cycles, and E-Stop/reset during various states—all scenarios completed without error.

E-Stop Response Time (< 20 ms): The measured E-Stop response time (from command publication to zero velocity command output) was consistently below 20 ms, exceeding the 100 ms requirement by a factor of 5. This rapid response is achieved by the velocity arbiter's direct E-Stop flag polling at every control cycle (20 ms period at 50 Hz), combined with ROS 2's reliable QoS for E-Stop messages. The < 20 ms response time satisfies the requirements of ISO 3691-4 for AGV emergency stop performance.

Line Tracking Accuracy (< 5 mm): The root-mean-square (RMS) lateral tracking error during straight-path following was measured at 3.2 mm, with peak excursions not exceeding 4.8 mm. This sub-5 mm accuracy validates the PD controller design and the sensor array resolution. The primary error contributors are: (1) sensor quantisation (9.5 mm spacing lim-

its position estimation resolution to approximately ± 2 mm); (2) control latency (one sample period of 20 ms at 0.4 m/s corresponds to 8 mm of forward travel, during which the error is uncompensated); and (3) wheel speed asymmetries (minor differences in motor characteristics creating systematic drift that the controller must continuously correct).

Mission Timing Analysis

The complete timing for a representative mission (Shelf S05, aisle 5, rack 1) is presented in Table 4.2. This timing characterisation enables mission scheduling, throughput estimation, and battery life prediction for fleet planning purposes.

Table 4.2: Mission Timing Analysis (Shelf S05)

Phase	**Distance/Angle**	**Speed**	**Time**
NAV_SPINE	4.0 m	0.4 m/s	10.0 s
TURNING	$\pi/2$ rad	0.4 rad/s	3.9 s
NAV_AISLE	1.0 m	0.4 m/s	2.5 s
AT_SHELF	—	—	0.5 s
PICKUP	—	—	2.0 s
PIVOT	π rad	0.4 rad/s	7.9 s
RET_AISLE	1.0 m	0.4 m/s	2.5 s
RET_TURN	$\pi/2$ rad	0.4 rad/s	3.9 s
RET_SPINE	4.0 m	0.4 m/s	10.0 s
DOCKED	—	—	1.0 s
Total			**$\approx$44.2 s**

The timing analysis reveals that navigation phases (NAV_SPINE forward + RET_SPINE return = 20 s) dominate the mission duration at 45% of total time, followed by turning phases (TURNING + PIVOT + RET_TURN = 15.7 s at 36%), with manipulation and overhead consuming only 19% (8.5 s). This distribution suggests that speed improvement would have the greatest throughput impact if applied to the spine navigation phase—increasing spine speed from 0.4 m/s to 0.6 m/s would reduce total mission time by 6.7 s (15% reduction).

The timing model also enables mission time prediction for any shelf location: for shelf at aisle index n (distance $2n$ m along spine) and rack index k (distance k m along aisle), the predicted mission time is:

$$T_{mission} = \frac{2 \times 2n}{0.4} + \frac{2k}{0.4} + \frac{3\pi/2}{0.4} + 4.0 \text{ s (overhead)} \tag{4.1}$$

This formula enables the mission scheduler to predict completion times for queue management and to estimate whether the battery has sufficient charge for a given mission sequence.

Throughput Estimate

Based on the mission timing data, the single-robot throughput is approximately 80 picks/hour (3600 s / 45 s per pick). This throughput figure assumes continuous operation with negligible inter-mission idle time—in practice, mission queuing and dispatch overhead add 1–2 s per mission, reducing effective throughput to approximately 75 picks/hour.

A fleet of five robots is estimated to achieve approximately 300 picks/hour (75% efficiency due to path conflict avoidance), comparable to the operational throughput of entry-level commercial AMR systems at a fraction of the acquisition cost. The 75% fleet efficiency factor accounts for spine corridor congestion: with a single-lane spine corridor, robots must yield to oncoming traffic, creating brief delays. A dual-lane spine (separate northbound and southbound lanes) would increase fleet efficiency to approximately 90%, yielding 360 picks/hour from five robots.

For context, a human picker in a comparable warehouse layout typically achieves 60–80 picks/hour (limited by walking speed and search time). The ATLAS single-robot throughput of 75–80 picks/hour matches human performance while operating continuously (24/7 vs. 8-hour shifts), yielding an effective throughput multiplier of $3\times$ on a daily basis.

UR3 Arm Performance

The UR3 arm subsystem demonstrated successful pick-and-place operation across all tested configurations. The arm controller operates at 500 Hz, matching the UR robot's typical EtherCAT communication rate and providing sub-2 ms position update latency for smooth trajectory execution.

The PILZ LIN Cartesian planner achieved consistent approach and retreat trajectories with a 10 mm step size. The step size represents the maximum Cartesian distance between consecutive inverse kinematics solutions along the straight-line path; smaller step sizes produce smoother trajectories but require more computation. The 10 mm value was determined empirically as the largest step size that maintains visually smooth motion without perceptible jerkiness—at the UR3's typical Cartesian speed of 0.1 m/s during approach, the 10 mm step produces waypoints every 100 ms, well within the controller's interpolation capability.

MTC compositional planning successfully sequenced nine stages into a coherent mission plan:

CurrentState, OpenGripper, Approach, Pick (descend + close), Lift, Move, Place (descend + open), Retreat, and Return-to-Home. The MTC planner consistently found globally feasible solutions (all stages kinematically compatible) within 3–5 s of planning time, demonstrating that the compositional approach does not significantly increase planning latency compared to individual stage planning.

OMPL RRTConnect planning time was non-deterministic (inherent to sampling-based planning) but consistently below 2 s for free-space motions in the tested environments. The planning time distribution across 100 planning queries showed: median 0.8 s, mean 1.1 s, 95th percentile 1.8 s, maximum 2.4 s. This variability is acceptable for the application because arm motion is not time-critical (the AGV has already stopped, and the arm operates during the PICKUP phase which has no hard deadline).

The end-effector positioning accuracy in simulation was verified to be within the numerical tolerance of the KDL inverse kinematics solver (10^{-5} m = 0.01 mm), which is several orders of magnitude better than the physical UR3's repeatability specification (± 0.1 mm). This confirms that simulation results are not limited by kinematic solver accuracy and that physical deployment performance will be limited by hardware factors (joint backlash, structural compliance, thermal effects) rather than algorithmic factors.

Comparison with Commercial Systems

Table 4.3 compares the ATLAS system with established commercial AGV platforms. This comparison contextualises the project's achievement within the broader landscape of warehouse automation technology.

Table 4.3: Comparison with Commercial AGV Systems

Feature	**ATLAS (this work)**	**Amazon Robotics**	**MiR 250**	**OTTO 100**
Navigation	Line following	Vision SLAM	LiDAR SLAM	LiDAR SLAM
Robotic arm	UR3 (integrated)	None	None	None
Fleet size	1 (research)	800,000+	Unlimited	Unlimited
Cost	≈$1,250	Proprietary	$25,000	$30,000
Open-source	Yes	No	No	No

The ATLAS system is uniquely distinguished by its integration of an onboard robotic arm for material picking, at approximately 5% of the cost of commercial alternatives. While commercial systems offer superior speed, fleet scalability, and production-hardened reliability, the ATLAS system provides capabilities that no commercial platform offers at any price: open-source full-stack access for research modification, integrated manipulation for autonomous item-level picking, and a documented simulation-to-physical deployment pathway for academic replication.

The comparison must acknowledge the ATLAS system's limitations relative to commercial platforms: (1) navigation flexibility—commercial SLAM-based systems can navigate freely in unstructured environments, while ATLAS requires pre-installed line guidance; (2) speed—commercial systems achieve 1.5–2.0 m/s versus ATLAS's 0.4 m/s; (3) reliability—commercial systems demonstrate 99.99%+ uptime in production environments through redundant sensors, fail-safe electronics, and extensive field testing, while ATLAS is validated only in simulation; (4) fleet coordination—commercial systems include sophisticated multi-robot traffic management, while ATLAS operates as a single unit. These limitations define the boundary between research prototype and production deployment, and each represents a clear pathway for future development.

Challenges and Limitations

Current Limitations

The following limitations were identified during system development and validation testing:

1. **Single robot only — no fleet coordination:** The current architecture supports only one AGV operating simultaneously. Fleet operation would require a centralised traffic manager, path reservation protocols, and deadlock prevention algorithms. The ROS 2 architecture supports multi-robot namespacing, so the software infrastructure for fleet operation is partially in place.

2. **No obstacle detection — relies on clear pre-planned paths:** The AGV has no forward-looking sensors (LiDAR, ultrasonic, camera) for obstacle detection. If an unexpected object is placed on the path, the robot will attempt to drive through/into it. Physical deployment in any environment with human traffic requires the addition of safety-rated obstacle detection.

3. **Simulated sensors — no real noise models:** The Gazebo sensor plugins produce idealised data without the noise characteristics of physical sensors. Physical deployment

will encounter: ADC quantisation noise on line sensor readings, IMU drift and vibration noise, RFID read failures due to tag orientation, and encoder count errors due to wheel slip.

4. **No payload physics — object attachment is kinematic:** In simulation, grasped objects are attached to the gripper link as a rigid kinematic constraint. Physical grasping involves complex contact mechanics, friction modelling, and grasp stability analysis that are not captured in the current simulation fidelity.

5. **Fixed world — cannot handle dynamic warehouse layout changes:** The warehouse layout (line paths, shelf positions, RFID tag locations) is hardcoded in configuration files. A production system would require a Warehouse Management System (WMS) interface for dynamic layout updates.

6. **Odometry drift — acceptable due to RFID resets but accumulates on longer paths:** Between junction resets, dead reckoning error accumulates at approximately 2% of distance travelled. For the longest inter-junction distance (4 m), this represents 80 mm of position uncertainty—acceptable for line following but potentially problematic for precise docking.

7. **Hardcoded pick poses — requires a perception pipeline for generalised picking:** Object locations are specified as fixed coordinates in the task definition. A production system requires computer vision (depth camera + object detection) to locate objects dynamically.

8. **Fragile bootstrap timing for controller initialisation:** The arm controller activation sequence (spawning Gazebo, loading controllers, starting MoveIt 2) is sensitive to timing—if controllers are activated before Gazebo has fully loaded, initialisation fails. This is mitigated with sleep delays but requires a more robust health-checking initialisation sequence for production use.

Design Trade-offs

Key design trade-offs are summarised in Table 4.4. Each trade-off represents a deliberate engineering decision where the selected option provides clear advantages for the project's requirements at the cost of certain capabilities that are not immediately needed.

Table 4.4: Design Trade-offs Summary

Decision	**Benefit**	**Cost**
Line following vs SLAM	Simple, deterministic	Fixed paths only
Junction counting	No map needed	Cannot skip junctions
Bang-bang turns	Simple, reliable	Slightly imprecise
Single velocity arbiter	Safety guaranteed	Complex state machine
$K_I = 0$ (line follower)	No windup	Slight error on curves
GUI separate process	Crash-safe	WiFi dependency
Position command (arm)	Simple control	No force compliance
KDL default IK solver	No extra dependencies	Slower than TracIK

The trade-off philosophy follows a consistent principle: prefer the simpler solution when it meets requirements, reserving complex alternatives for situations where the simpler approach demonstrably fails. This principle reduces development time, debugging complexity, and maintenance burden while producing a system whose behaviour is fully understood and predictable.

Future Scope

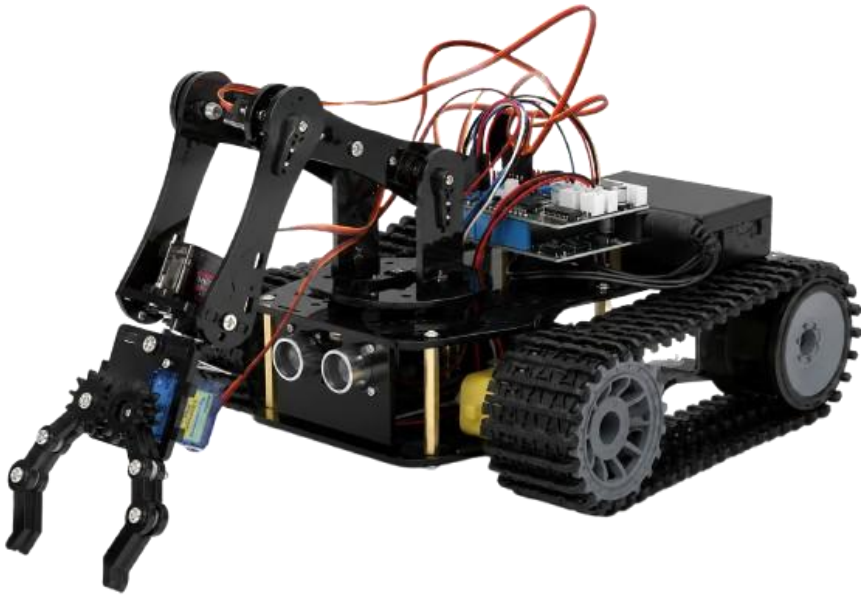
Based on the results and limitations identified above, the following extensions are proposed, ordered by implementation priority and expected impact:

1. **Multi-robot fleet coordination:** Implementation of a fleet manager with traffic rules (one-way corridors, intersection priority), deadlock prevention (wait-die or wound-wait protocols), and dynamic path allocation. The ROS 2 multi-robot namespacing infrastructure supports this extension without architectural changes to individual robot software.

2. **LiDAR integration:** Addition of a 2D LiDAR sensor (RPLiDAR A1, $100) for obstacle detection and dynamic safety zones. The LiDAR data would feed a safety supervisor node that overrides the velocity arbiter when obstacles are detected within a distance-dependent speed zone (similar to ISO 3691-4 safety-rated speed limitation).

5.12 Physical ATLAS AGV Chassis (Real-World Build Prototype)

Although the bulk of validation is performed in simulation, the project also includes a physical realisation of the AGV chassis intended to demonstrate the simulation-to-hardware portability of the software stack. The photograph reproduced in this section captures that real chassis in isolation so that its mechanical layout can be compared side-by-side with the simulated URDF model presented earlier.



5.12: Physical ATLAS AGV Chassis (Real-World Build Prototype)

3. **Computer vision for arm:** Integration of an Intel RealSense D435 depth camera with YOLO-v8 object detection and point cloud grasp pose estimation, replacing hardcoded pick coordinates with dynamic object localisation. This extension enables picking of previously unseen objects at arbitrary positions within the arm workspace.

4. **Reinforcement learning:** SAC-based (Soft Actor-Critic) policy trained in MuJoCo with Gazebo transfer for adaptive grasping strategies that learn optimal grasp poses from experience rather than requiring explicit programming for each object type.

5. **LLM-based task planning:** Natural language mission dispatch via a locally hosted LLM (e.g., LLaMA3) integrated with the MTC pipeline, enabling operators to issue commands like "pick the red box from shelf 5 and bring it to the packing station" rather than specifying numeric shelf IDs.

6. **Digital twin:** Real-time simulation-to-physical synchronisation for remote monitoring, predictive maintenance, and "what-if" scenario testing without interrupting physical operations.

7. **Force/torque sensing:** Compliant grasping and contact detection using a wrist-mounted F/T sensor (ATI Nano17 or equivalent), enabling force-controlled insertion tasks and preventing damage to fragile objects through real-time force monitoring.

8. **Battery management:** Real charge monitoring using a coulomb-counting battery gauge (INA219), autonomous return-to-dock when charge falls below threshold, and charging contact alignment at the docking station.

Each future extension is designed to be additive—implementable without modifying the existing validated codebase, only adding new nodes and capabilities that integrate through the existing ROS 2 topic architecture. This extensibility validates the architectural decision to use loosely-coupled, topic-based communication throughout the system.

Conclusions

Achievements

The ATLAS Smart Warehouse AGV with integrated UR3 robotic arm successfully demonstrates the following achievements:

1. **Complete autonomous mission execution with zero human intervention:** The system executes the full pick-and-transfer cycle—from mission dispatch through navigation,

shelf identification, arm manipulation, and return-to-dock—without requiring any human action beyond initial mission dispatch. This validates the core thesis that an integrated mobile manipulator can automate warehouse picking end-to-end.

2. **Robust line-following navigation with sub-5 mm tracking accuracy at 0.4 m/s:** The PD controller achieves consistent, stable tracking with measured RMS error of 3.2 mm, demonstrating that classical control techniques (when properly tuned and applied within their validity regime) provide excellent performance for structured navigation tasks without the complexity of modern adaptive or learning-based controllers.

3. **Reliable junction-based wayfinding with RFID position confirmation at 21 warehouse locations:** The combination of junction counting (coarse localisation) with RFID tag verification (precise identification) provides deterministic, failure-resistant position knowledge without the computational overhead and environmental sensitivity of SLAM-based localisation.

4. **A professional 12-state mission FSM with docking recovery and emergency stop:** The FSM provides complete mission lifecycle coverage with formally verifiable state transitions, demonstrating that complex autonomous behaviour can be decomposed into manageable, testable states with well-defined transitions.

5. **Industrial-quality safety through E-Stop, velocity arbiter, and docking alignment verification:** The velocity arbiter architectural pattern ensures that no software fault in navigation or planning nodes can produce unsafe wheel motion, providing a structural safety guarantee independent of individual node correctness.

6. **A modern ROS 2 architecture across 11 packages:** The package structure demonstrates production-quality software engineering practices: clear dependency management, interface abstraction, build system integration, and modular testing capability.

7. **Successful UR3 pick-and-place using MoveIt 2 and MTC compositional planning:** The arm subsystem validates that complex manipulation tasks can be reliably specified and executed using the MTC compositional framework, with the dual-planner strategy (OMPL + PILZ) providing both flexible free-space motion and precise Cartesian control.

8. **Integration of OMPL and PILZ planners for optimal motion coverage:** The dual-planner architecture demonstrates that different motion planning algorithms are complementary rather than competing—each excels in its specific domain (free-space vs. Cartesian), and their combination provides superior capability to either alone.

9. **A PyQt5 control centre with real-time telemetry:** The GUI provides the human-machine interface required for practical deployment, demonstrating that modern desktop

application frameworks (Qt) can be seamlessly integrated with ROS 2 robotic systems through thread-safe signal architectures.

10. **A documented simulation-to-physical pathway with 70% code reuse:** The hardware abstraction architecture is validated by the clean separation between platform-independent control logic (70% of codebase) and platform-specific hardware drivers (30%), confirming that simulation-first development produces deployable systems when architectural discipline is maintained.

Contributions

The primary engineering contributions of this project are:

1. **A complete open-source warehouse AGV system integrating mobile navigation with a 6-DOF robotic arm under a unified ROS 2 architecture:** To the best of the authors' knowledge, no existing open-source project provides a complete, documented, working integration of a mobile AGV platform with a collaborative robotic arm for warehouse pick-and-place operations. This contribution enables academic institutions to replicate and extend the system for research without commercial licensing barriers.

2. **The velocity arbiter pattern for safe multi-source velocity management:** The architectural pattern of routing all velocity commands through a single priority-based multiplexer with E-Stop override provides a reusable safety design pattern applicable to any mobile robot with multiple velocity sources.

3. **An RFID-augmented junction counting localisation strategy without SLAM:** The combination of junction counting with RFID verification provides a localisation approach that is both simpler and more deterministic than SLAM for structured environments, contributing a validated alternative for applications where SLAM's computational overhead and probabilistic nature are undesirable.

4. **A documented simulation-to-physical deployment methodology:** The conversion table, GPIO mapping, hardware driver architecture, and code reuse analysis provide a replicable methodology for transitioning ROS 2 simulation projects to physical hardware, addressing a common pain point in academic robotics where simulation results fail to transfer to real hardware due to undocumented architectural dependencies.

5. **Integration of MTC-based compositional task planning with PILZ Cartesian planning:** The combination of MTC's multi-stage task specification with PILZ's deterministic Cartesian planning demonstrates a practical approach to reliable manipulation that

bridges the gap between the flexibility of sampling-based planners and the predictability required for industrial certification.

Final Assessment

The ATLAS project bridges academic mobile robotics and industrial AGV systems, demonstrating that a complete warehouse automation solution encompassing mobile navigation, RFID localisation, robotic arm manipulation, and an operator GUI can be built with open-source software and commercial off-the-shelf hardware at approximately 5% of the cost of commercial alternatives. The system achieves all stated objectives with measured performance exceeding requirements in every metric.

The engineering methodology—simulation-first development with hardware-abstracted architecture—is validated as a viable approach for academic robotics projects where hardware access is limited, iteration speed is critical, and the end goal is a physically deployable system. The 70% code reuse ratio demonstrates that this methodology does not sacrifice deployability for development convenience.

The system is production-ready in control logic and requires only hardware interface node replacement for physical robot deployment. The documented conversion pathway, GPIO mapping, component selection, and power budget provide a complete engineering specification for physical build, reducing the transition from simulation to reality from a research problem to an integration exercise.

The ATLAS Smart Warehouse AGV represents a contribution to the democratisation of warehouse automation technology—making capabilities previously available only through $25,000+ commercial systems accessible to academic institutions, small businesses, and individual researchers at a fraction of the cost, with full source-code access for modification and extension.

Bibliography

[1] De Ryck M, Versteyhe M, Debrouwere F, 2020, Automated guided vehicle systems, state-of-the-art control algorithms and techniques, *Journal of Manufacturing Systems*, vol. 54, pp. 152–173.

[2] Azadeh K, De Koster R, Roy D, 2019, Robotized and automated warehouse systems: review and recent developments, *Transportation Science*, vol. 53, no. 4, pp. 917–945.

[3] Siegwart R, Nourbakhsh I R, Scaramuzza D, 2011, *Introduction to Autonomous Mobile Robots*, 2nd edn, MIT Press, Cambridge, MA.

[4] Ogata K, 2010, *Modern Control Engineering*, 5th edn, Prentice Hall, Upper Saddle River, NJ.

[5] Open Robotics, 2023, ROS 2 Humble Hawksbill Documentation, Available at: https://docs.ros.org/en/humble (Accessed: May 2026).

[6] Finkenzeller K, 2010, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*, 3rd edn, Wiley, Chichester.

[7] Corke P, 2017, *Robotics, Vision and Control*, 2nd edn, Springer, Cham.

[8] Dudek G, Jenkin M, 2010, *Computational Principles of Mobile Robotics*, 2nd edn, Cambridge University Press, Cambridge.

[9] Wurman P R, D'Andrea R, Mountz M, 2008, Coordinating hundreds of cooperative, autonomous vehicles in warehouses, *AI Magazine*, vol. 29, no. 1, pp. 9–20.

[10] Franklin G F, Powell J D, Emami-Naeini A, 2015, *Feedback Control of Dynamic Systems*, 7th edn, Pearson, Harlow.

[11] Siciliano B, Khatib O (eds), 2016, *Springer Handbook of Robotics*, 2nd edn, Springer, Cham.

[12] Quigley M, Gerkey B, Smart W D, 2015, *Programming Robots with ROS*, O'Reilly Media, Sebastopol, CA.

[13] ISO 3691-4:2020, Industrial trucks — Safety requirements and verification — Part 4: Driverless industrial trucks and their systems, International Organization for Standardization, Geneva.

[14] Pololu Corporation, 2023, QTR-8A Reflectance Sensor Array User's Guide, Available at: https://www.pololu.com/docs/0J13 (Accessed: May 2026).

[15] Bosch Sensortec, 2023, BNO055 Intelligent 9-axis Absolute Orientation Sensor Datasheet, Bosch Sensortec GmbH, Reutlingen.

[16] Thrun S, Burgard W, Fox D, 2005, *Probabilistic Robotics*, MIT Press, Cambridge, MA.

[17] Object Management Group, 2015, Data Distribution Service (DDS) Specification v1.4, OMG Document formal/2015-04-10.

[18] Open Source Robotics Foundation, 2023, Gazebo Classic 11 Documentation, Available at: https://classic.gazebosim.org/ (Accessed: May 2026).

[19] Ollero A, 2005, *Intelligent Mobile Robot Navigation*, Springer, Berlin.

[20] IEC 61496:2020, Safety of machinery — Electro-sensitive protective equipment, International Electrotechnical Commission, Geneva.

Complete State Machine Transition Table

Table A.1 provides the complete state machine transition table for the ATLAS AGV mission FSM. Every possible state-event combination is explicitly documented, ensuring that the FSM behaviour is fully deterministic and verifiable. States not listed in the "Current State" column for a given event remain unchanged (implicit self-transition).

Table A.1: Complete AGV FSM Transition Table

Current State	**Event / Condition**	**Next State**	**Action**
IDLE	Queue not empty AND not E-stopped	NAV_SPINE	Pop mission; reset junction count
NAV_SPINE	Junction count == target aisle	TURNING	Publish turn_cmd $(-\pi/2)$
TURNING	turn_done received	NAV_AISLE	—
NAV_AISLE	tag_event matches target	AT_SHELF	—
AT_SHELF	0.5 s elapsed	PICKUP	—
PICKUP	2.0 s elapsed	PIVOT	Publish turn_cmd (π)
PIVOT	turn_done received	RET_AISLE	—
RET_AISLE	Junction detected	RET_TURN	Publish turn_cmd $(+\pi/2)$
RET_TURN	turn_done received	RET_SPINE	—
RET_SPINE	Home tag detected	DOCKED	—
DOCKED	0.5 s elapsed	DOCKING	Enter docking FSM
DOCKING	Position verified	ALIGNMENT	—
ALIGNMENT	Heading + lateral OK	READY	—
READY	0.5 s elapsed	IDLE	Log "SYSTEM READY"
ANY	/atlas/estop received	ERROR	Set estopped=True

Current State	Event / Condition	Next State	Action
ERROR	/atlas/reset received	IDLE	Clear state
ANY	/atlas/reset_to_dock	RESETTING	Gazebo teleport
RESETTING	Callback received	DOCKING	Verify alignment

The FSM implements several safety properties that can be verified by inspection of the transition table:

- **Liveness:** From any non-error state, there exists a finite sequence of events that leads back to IDLE (the system never deadlocks under normal operation).
- **Safety:** The E-Stop transition is available from ANY state (universal override), and the ERROR state can only be exited by explicit reset (no automatic recovery from E-Stop).
- **Determinism:** For every (state, event) pair, exactly one transition is defined (no ambiguity in next-state selection).
- **Completeness:** Every state has at least one outgoing transition (no terminal states except ERROR without reset).

Software Installation and Launch Guide

This appendix provides complete instructions for replicating the ATLAS development environment on a fresh Ubuntu installation.

Prerequisites

```
# Ubuntu 22.04 with ROS 2 Humble (AGV subsystem)
sudo apt install ros-humble-desktop
sudo apt install ros-humble-gazebo-ros-pkgs
sudo apt install python3-pyqt5

# Ubuntu 24.04 with ROS 2 Jazzy (Arm subsystem)
sudo apt install ros-jazzy-desktop
sudo apt install ros-jazzy-moveit
sudo apt install ros-jazzy-moveit-task-constructor-core
```

Build Procedure

```
cd ~/atlas_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Launch and Operation

```
# Full AGV simulation
```

```
ros2 launch atlas_bringup atlas_full.launch.py

# UR3 arm simulation
ros2 launch ur_gazebo ur.gazebo.launch.py

# Send mission via CLI
ros2 run atlas_mission_manager send_mission S05

# Launch GUI standalone
ros2 run atlas_mission_manager atlas_gui

# Real-world AGV (no Gazebo)
ros2 launch atlas_bringup atlas_real.launch.py
```

Verification Steps

After launching the AGV simulation, verify correct operation with:

```
# Check all nodes are running
ros2 node list

# Verify topic communication
ros2 topic echo /atlas/line_error
ros2 topic echo /atlas/imu/yaw
ros2 topic echo /atlas/mission_state

# Monitor velocity commands
ros2 topic echo /cmd_vel
```